

浙江大学

课程设计报告

课程名称： 数字系统

任课老师： 李宇波

课程设计名称： 智能陪伴小车

完成日期： 2026 年 6 月 29 日

小组： 第 3 小组

小组成员： 刘卓彦、郑一韬、李彦萱

目录

1	项目的目的和要求	3
1.1	项目目的	3
1.2	项目实施要求	3
2	原理与设计	4
2.1	外观设计	4
2.2	系统总体架构	4
2.3	PID 控制原理	5
2.3.1	算法原理	5
2.3.2	双环 PID 结构	5
2.4	超声波测距原理	6
2.4.1	HC-SR04 传感器工作原理	6
2.4.2	双路交替测距设计	7
2.5	PWM 电机驱动原理	7
2.5.1	H 桥驱动与 PWM 调速	7
2.5.2	方向控制逻辑	8
2.6	OLED 显示与 I2C 通信原理	8
2.6.1	SSD1306 OLED 显示屏	8
2.6.2	I2C 通信协议	8
2.7	情感化表情显示设计	9
2.8	多模式行为切换设计	9
2.9	语音识别与跨芯片异步通信	10
3	测试和分析	12
3.1	自动跟随功能测试与分析	12
3.2	语音控制模式切换功能测试与分析	13
3.3	OLED 显示功能测试与分析	13
3.4	实践过程中的优化	14
3.4.1	语音通信架构方案的迭代	14
3.4.2	总线通信协议的精简	14
4	项目成果	14
4.1	实物总览	14
4.2	实现的功能	15
4.3	设计的课程知识运用	16
4.3.1	运控模块:	16
4.3.2	表情控制:	17

4.4	设计的复杂性	17
4.5	设计特点与创新	17
5	总结	18
5.1	设计完成情况	18
5.2	不足与展望	18
5.3	心得	19
A	代码结构	19
A.1	模块结构	19
B	完整代码	20
B.1	control.v	20
B.2	OLED/iic.v	36
B.3	OLED/font_data.v	48
B.4	OLED/Oled_Top.v	66
B.5	OLED/Oled_Init.v	72
B.6	OLED/Oled_Show_control.v	74
B.7	OLED/Oled_Show_Info.v	77
B.8	OLED/Oled_Clear.v	79
B.9	OLED/Oled_On.v	81
B.10	OLED/topcontrol.v	83

1 项目的目的和要求

1.1 项目目的

在智慧校园的建设从“基础设施数字化”向“以人为本的智能化”转型的背景下，学生心理健康日益受到重视。传统的心理咨询存在门槛高、非实时等问题。一款能够部署在宿舍、图书馆或心理咨询室的“智能陪伴小车”，能通过低成本、高效率的方式提供情感慰藉与日常交互，成为校园智慧生态中温情的一环。

本课程设计旨在设计并实现一台基于 FPGA 驱动的智能陪伴小车，通过 Verilog 完成全部控制逻辑，在 FPGA 上实现“传感器数据采集 → 控制算法运算 → 执行器驱动 → 人机交互”的完整闭环。我们设计的小车具备以下四项能力：

- (1) **自动跟随：**通过双路 HC-SR04 超声测距模块实时检测前方目标距离，利用双环 PID 控制器实现对人的自动跟随。
- (2) **语音对话：**我们的小车能够通过麦克风模块接收用户指令，并根据不同情感进行回应，实现简单的问答交互。
- (3) **多种行为：**智能陪伴小车能够根据场景和情感判断完成跟随、转弯、停止、自动跟随、前进与后退连接、原地旋转等 4 种主要运行模式之下多样的行为，通过对话返回的情感判断进行模式切换。
- (4) **情感交互：**我们采用 SSD1306 OLED 显示屏（128×64，I2C 接口），根据当前工作模式实时显示不同的面部表情（开心/平静/伤心）。
- (5) **电机驱动控制：**输出 PWM 信号及 IN1/IN2 方向信号，使用 L298N 电机驱动模块，驱动双路直流减速电机。

1.2 项目实现要求

- (1) 综合利用数字系统课程相关知识，充分掌握并运用课内所学的 Verilog 语言、时序逻辑、模块化设计等知识和技能。
- (2) 智能陪伴小车功能要主要采用 Verilog 语言实现，利用 EGO1 开发板运行。
- (3) 各模块之间接口清晰、职责分明，便于独立仿真调试与系统联调。
- (4) 超声波测距刷新周期约为 50 ms，采用双环 PID 算法，期望静态跟随距离稳定在约 50 cm，转向环能根据左右距离差实时调整。
- (5) OLED 显示需支持模式切换时的自动复位和表情切换。
- (6) 各模块之间电压转换需符合各器件的工作电压要求。

2 原理与设计

2.1 外观设计

我们基于功能实现要求设计了三层小车，除了基本车架之外，电子元器件包含 SSD1306 OLED 显示屏、HC-SR04 超声测距模块、电压转换器、L298N 电机驱动模块、直流减速电机、麦克风模块、EGO1 FPGA 开发板、ASRPRO 语音识别芯片以及电源模块。小车外观设计如下图所示：

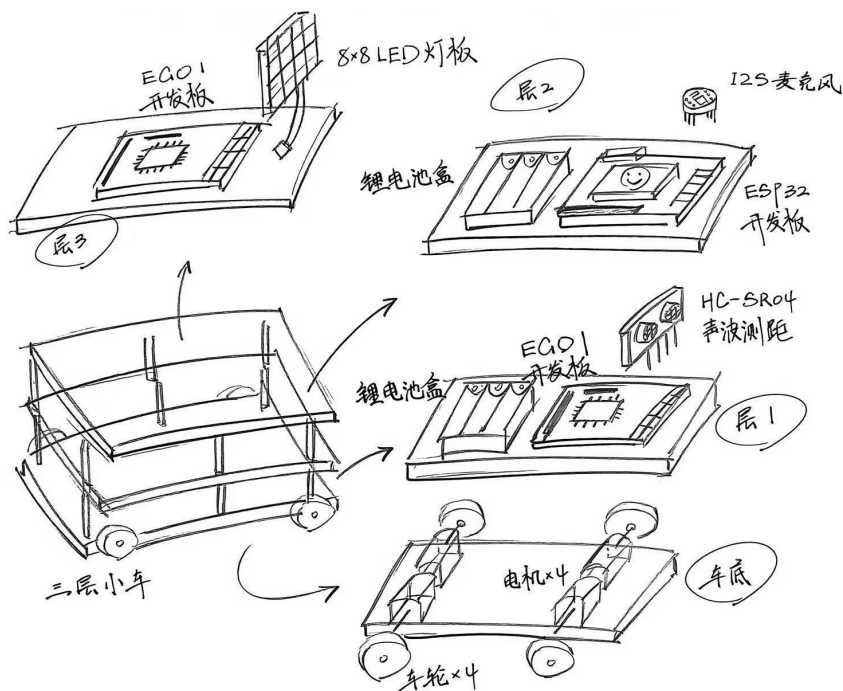


图 1: 外观设计图

2.2 系统总体架构

我们在项目的代码设计中采用模块化设计，整体架构如下图所示。智能陪伴小车由运控系统、OLED 显示系统、语音交互系统组成。

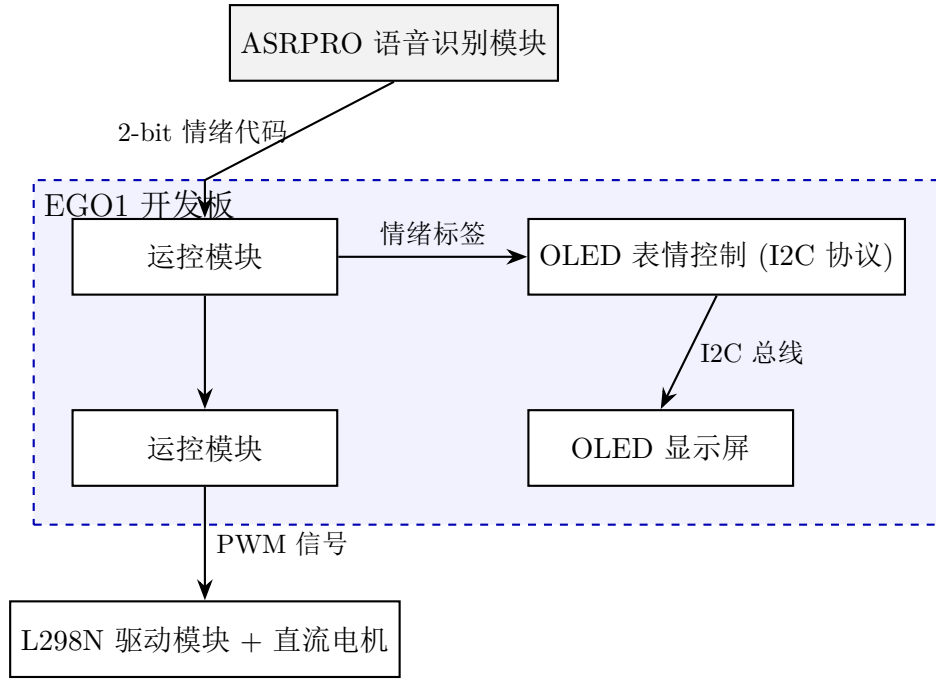


图 2: 基于 FPGA 的智能陪伴小车异构控制系统总体架构图

2.3 PID 控制原理

2.3.1 算法原理

我们的智能陪伴小车在运动控制模块设计时采用 PID 控制算法，其数学表达为：

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1)$$

其中 $e(t)$ 为偏差信号， K_p 、 K_i 、 K_d 分别为比例、积分、微分系数。由于 FPGA 是离散数字系统，式 1 需离散化后在硬件中实现。我们在算法设计时采用定点数运算以节省硬件资源，即内部计算精度保留 8 位小数位，输出时通过右移 8 位恢复实际量纲：

$$\begin{aligned} e_k &= \text{target} - \text{actual} \\ P_k &= K_p \cdot e_k \\ I_k &= I_{k-1} + e_k \\ D_k &= K_d \cdot (e_k - e_{k-1}) \\ u_k &= \frac{P_k + K_i \cdot I_k + D_k}{256} \end{aligned} \quad (2)$$

2.3.2 双环 PID 结构

为实现小车对人的平滑跟随，我们在设计中采用双环 PID 结构。

速度环：以 50cm 对应的超声波计数值为目标值，以两路超声波平均距离计数值为反馈量，输出基础速度信号 v_{speed} 。当小车距离目标过远时加速接近，过近时减速甚至后退。

转向环：以 0 为目标值（代表左右距离相等），以两路超声波距离差为反馈量，输出转向修正信号 v_{turn} 。当目标偏左时，右轮加速、左轮减速/反转，使小车向右修正方向。

理论上，两环输出经**差速混控公式**合成左右轮占空比：

$$\begin{cases} \text{left_duty} = \text{speed_ctrl} + \text{turn_ctrl} \\ \text{right_duty} = \text{speed_ctrl} - \text{turn_ctrl} \end{cases} \quad (3)$$

这种差速驱动方式使得转向修正不影响整体前进速度，保证了跟随过程的平顺性。两个 PID 控制器实例的参数配置如表 1 所示。速度环的比例系数和积分系数均高于转向环，这是因为距离跟随对响应速度要求更高，而转向修正需要更保守的参数以避免方向振荡。

表 1: PID 控制器实例化参数

实例	用途	K_p	K_i	K_d	输出范围
u_speed_pid	速度控制	512	128	256	± 200
u_turn_pid	转向控制	384	64	192	± 150

2.4 超声波测距原理

2.4.1 HC-SR04 传感器工作原理

HC-SR04 是一种常用的非接触式距离测量模块，工作频率为 40kHz，测距范围为 2cm-400cm（但是在实际应用中没有那么大）。其工作流程如下：

- (1) 主控器向 Trig 引脚发送一个时长 $\geq 10\mu\text{s}$ 的高电平触发脉冲。
- (2) 模块内部自动发出 8 个 40kHz 的超声波脉冲，同时将 Echo 引脚拉高。
- (3) 超声波在空气中传播，遇到障碍物后反射回来。
- (4) 模块接收到回波后将 Echo 引脚拉低。
- (5) 主控器通过测量 Echo 高电平的持续时间 t_{echo} 来计算距离：

$$d = \frac{v_{\text{声}} \cdot t_{\text{echo}}}{2} \quad (4)$$

2.4.2 双路交替测距设计

为实现对目标方位的感知，我们在小车头部左右各安装一个 HC-SR04 超声波传感器。FPGA 通过 FSM 交替触发和测量两路传感器：

```

IDLE → TRIG_LEFT → WAIT_ECHO_L → MEASURE_L → STORE_L
      → TRIG_RIGHT → WAIT_ECHO_R → MEASURE_R → STORE_R
      → WAIT_50MS → IDLE (循环)
  
```

关键设计要点：

- **触发脉冲宽度：**1000 个时钟周期为 $10\ \mu\text{s}$ ，满足 HC-SR04 对触发脉宽的最低要求。
- **超时保护：**等待回波和测量阶段均设置 3,000,000 个时钟周期（约 30 ms）的超时上限。若 Echo 信号在此时间内未出现或未结束，状态机自动跳转到下一阶段，避免因传感器故障导致系统死锁。
- **异步信号同步：**Echo 信号为异步输入，通过两级寄存器同步消除亚稳态，再通过边沿检测电路捕获上升沿和下降沿。
- **测量周期：**完整一轮左右交替测距约需 50ms，对应于 20 Hz 的控制更新频率。对于步行速度的跟随场景，该更新率满足实时性要求。

测距完成后计算两个关键信号供 PID 控制器使用：

$$\text{avg_cnt} = (\text{left_cnt} + \text{right_cnt}) \gg 2 \quad (\text{平均距离计数值}) \quad (5)$$

$$\text{diff_signed} = \text{right_cnt} - \text{left_cnt} \quad (\text{左右距离差}) \quad (6)$$

2.5 PWM 电机驱动原理

2.5.1 H 桥驱动与 PWM 调速

在制作智能陪伴小车时，我们采用了 L298N 双路 H 桥电机驱动模块控制两个直流减速电机。H 桥电路通过四个开关管的通断组合实现电机的正转、反转和制动。L298N 的每路 H 桥需要三个控制信号：ENA 和两个方向输入端（IN1、IN2）。

理论上脉宽调制可以通过调节单位时间内占空比来控制电机的平均电压，从而调节转速。PWM 参数设计如下：

- **载波频率：** $20\text{kHz} \left(\frac{100\text{MHz}}{5000} = 20\text{kHz} \right)$ 。
- **占空比分辨率：**8 位（0-255），经限幅后映射到 PWM 比较值为 0-4999。
- **计数范围：**0-4999，每个 PWM 周期将计数器值与占空比较值比较，生成 PWM 波形。

但是在实际的应用中，我们因为电压转换器不足没有采用占空比，利用正转和反转来实现转弯、转圈等功能。

2.5.2 方向控制逻辑

电机方向由 IN1/IN2 信号的组合控制，逻辑如表 2 所示。该逻辑将 PID 输出的有符号控制量直接映射为电机方向和转速：正 duty 值对应正转，负 duty 值对应反转，零值对应停止。

表 2: 电机方向控制逻辑（适配 L298N）

duty 符号	IN1	IN2	电机状态
duty > 0	1	0	正转（前进）
duty < 0	0	1	反转（后退）
duty = 0	0	0	停止（刹车）

2.6 OLED 显示与 I2C 通信原理

2.6.1 SSD1306 OLED 显示屏

为了更好地显示表情我们选用 1.3 寸 SSD1306 OLED 显示屏，通过 I2C 串行总线与 FPGA 通信。SSD1306 内部集成 GDDRAM，按页组织——共 8 页、每页 128 列 ×8 位。写入 GDDRAM 的数据直接映射到屏幕像素（1= 点亮，0= 熄灭）。

2.6.2 I2C 通信协议

I2C 是一种双线串行总线协议，仅需 SCL 和 SDA 两条信号线。在小车设计中，FPGA 作为 I2C 主机，OLED 作为从机，实现了一个完整的 I2C Master 控制器，特性如下：

- **SCL 频率：**约 200kHz，满足 SSD1306 标准模式和快速模式的要求。
- **协议完整性：**自动生成 START 和 STOP 条件，每字节传输后检测从机 ACK/NACK 应答。
- **SDA 双向控制：**通过三态门实现 SDA 的输入/输出切换，在 ACK 检测阶段释放总线由从机驱动。
- **错误处理：**从机未应答时输出 Error 信号，供上层模块处理。

I2C Master 状态机共约 58 个状态，完整覆盖一次写操作流程：START → 发送从机地址（7bit+W）→ ACK → 发送寄存器地址（8bit）→ ACK → 发送数据（8bit）→ ACK → STOP。

2.7 情感化表情显示设计

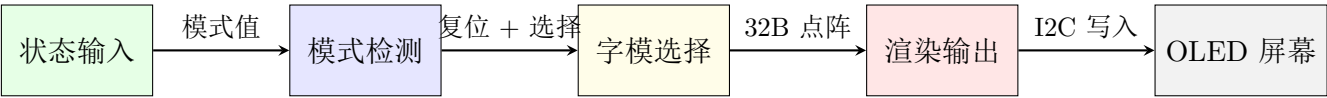


图 3: 情感化表情显示的信号通路

情感交互是我们的一个创新点，表情切换的信号通路如图 3 所示：语音交互后判断情感状态 → 检测变化并复位 OLED→ 根据 sw 值选择开心/平静/难过对应的字模数据组 → 将字模转换为 I2C 写命令序列 → 经 FPGA 发送到 OLED 屏幕。每套表情由我们自定义的点阵字符拼接而成，共同构建一张完整的表情画面。

2.8 多模式行为切换设计

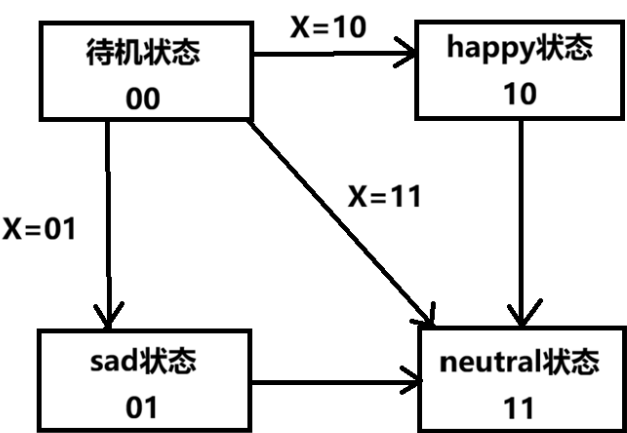


图 4: 状态转换图

系统的 4 种工作模式及对应行为如表 3 所示。其中 FOLLOW 为常态运行模式；STOP、FWD_REV 和 SPIN 为情感动作，执行完预定义动作序列后自动返回 FOLLOW。这一设计确保小车不会长时间停留在非跟随状态。

表 3: 小车 4 种运行模式定义

mode[1:0]	模式名称	行为描述	持续时间	OLED 表情
00	STOP	电机静止，PWM 输出置零	稳态	—
01	FOLLOW	PID 自动跟随目标	平静	正常
10	FWD_REV	前进 2s 与后退 2s 交替	共约 8s	难过
11	SPIN	左轮正转 + 右轮反转	10s	开心

模式切换由 car_follow 模块内部的 action_busy 标志位管理：当检测到模式切换边沿时，action_busy 置位并锁定当前动作直至执行完毕，期间新的模式切换请求被忽略。这一互锁机制有效防止了用户在动作执行中切换模式导致的控制信号冲突。

2.9 语音识别与跨芯片异步通信

我们在设计语音模块时先后采用两种方案：分别为基于 LLM 的多模态情感映射与本地化边缘语音芯片 ASRPRO。但是由于麦克风硬件识别准确率不高，尽管我们成功实现了调用大模型并结合输入语言返回恢复以及设定感情，我们最后还是采用了使用 ASRPRO 芯片的方案。

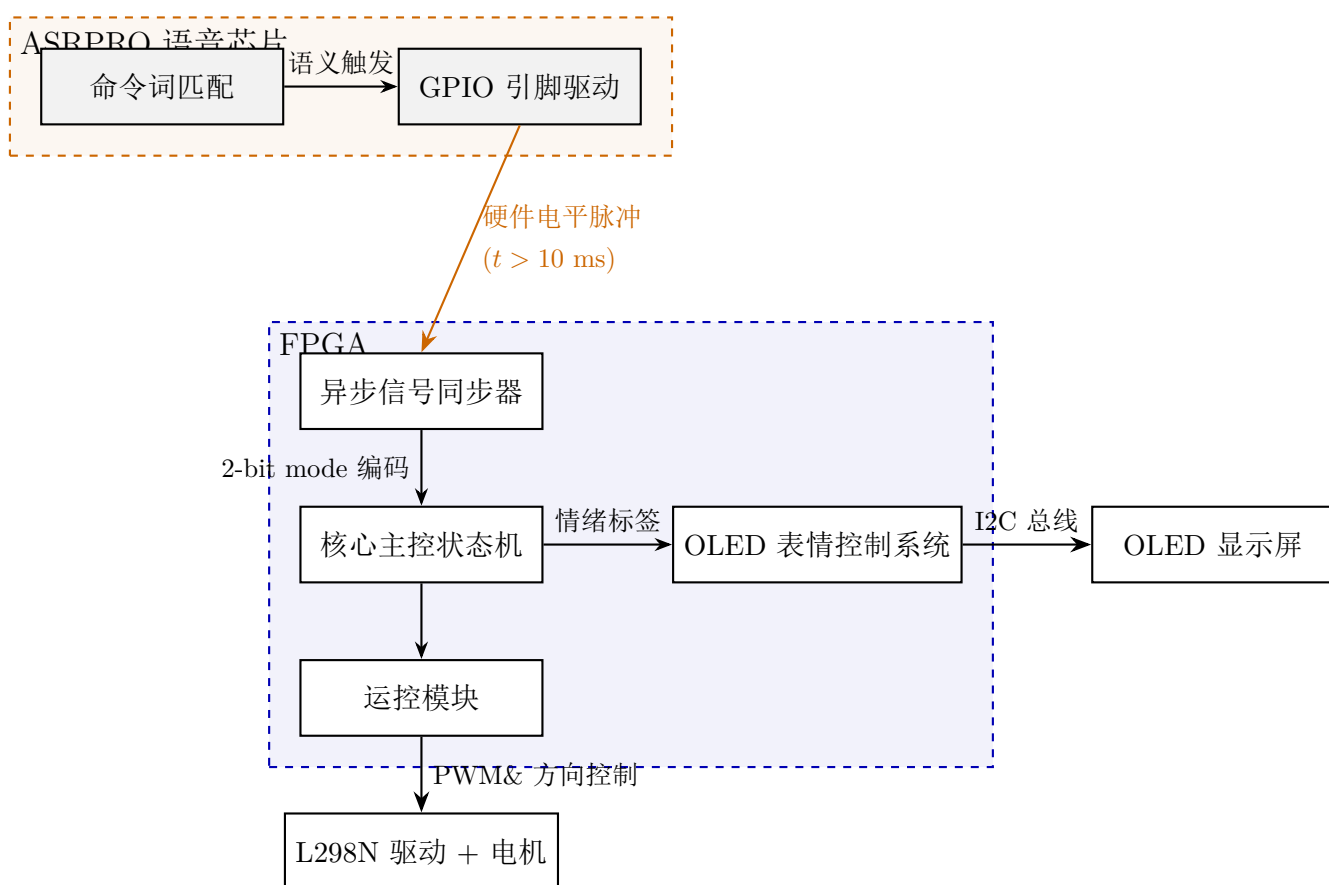


图 5: 包含语音识别交互与异步采样机制的系统总体架构原理图

下面主要对 ASRPRO 的原理与设计进行阐述：

- **边缘语音识别与语义映射原理：**ASRPRO 芯片内部集成了本地神经网络微型加速器，在经过特定唤醒词与命令词的固件烧录后，芯片可在本地完成对麦克风采集到的音频流的特征提取与匹配。当识别到有效语义后，芯片内部的 MCU 会将高级文本语义转换为特定引脚的电平跳变。

- **跨时钟域异步脉冲通信机制：**由于 ASRPRO 芯片内部核心时钟与 FPGA 主控系统时钟相互独立,为避免直接采用 UART 串口通信给 FPGA 状态机带来轮询开销与时序亚稳态风险，我们使用了电平脉冲宽度采样协议：ASRPRO 在识别到不同命令后，向 FPGA 的 GPIO 引脚发送特定时间窗口宽度的硬件电平脉冲。FPGA 端的接收控制模块在主频下对该引脚实施持续的两级寄存器打拍同步，以消除亚稳态。同时，在脉冲有效期间进行计数采样。若电平宽度满足预设阈值，则将该状态锁存进一个 2-bit 的情绪代码寄存器，从而在极低硬件资源消耗下实现了高抗干扰性的跨芯片数据传递。

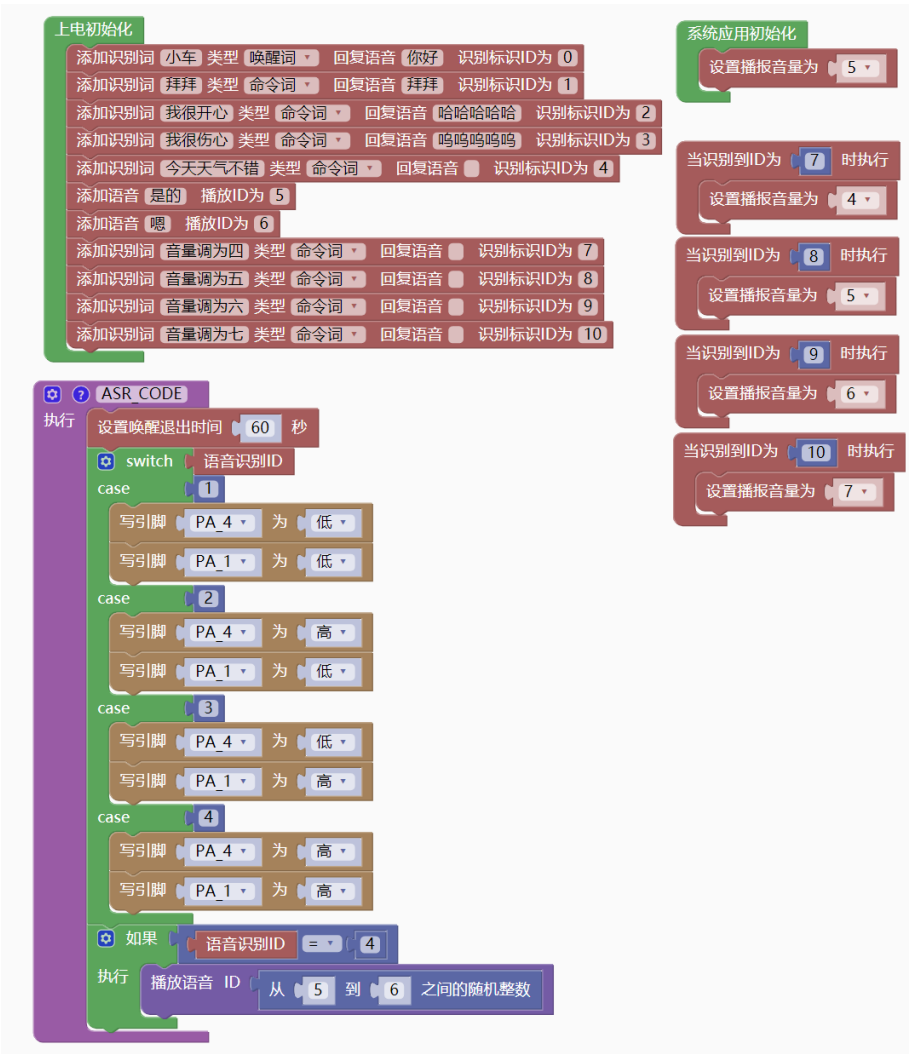


图 6: 语音模块代码

3 测试和分析

3.1 自动跟随功能测试与分析

在 FOLLOW 模式下对小车进行跟随性能测试。测试方法：以行人作为跟随目标，在小车前方以步行速度移动，观察小车跟随行为。

测试结果：

天崩开局，我们的小车因为电机驱动能力不足跑不动，只能把充电宝拿起来像遛电子宠物一样才行。开始人为遛之后继续天崩，这个小车在教学楼里乱跑，动不动就倒车，或者拐弯。经排查是因为超声波过于灵敏，但是到空旷的户外他又不太跑得动。只能找较为空的室内。最终在较理想的情况下，小车能实现即定功能。

尽管我们的双环 PID 结构在跟随场景中理论上控制效果非常好。但是因为硬件太差，优势被严重削弱。

3.2 语音控制模式切换功能测试与分析

我们将语音与运控进行联测，依次通过语音对话测试了 FOLLOW → SPIN → FOLLOW → FWD_REV → FOLLOW 的完整切换流程。

测试结果：

- **SPIN 模式：**切换到 SPIN 后，左轮正转、右轮反转，OLED 同步切换为开心表情。大约 10 秒后自动回到 FOLLOW 模式。
- **FWD_REV 模式：**切换到 FWD_REV 后，小车按”前进 2s 后退 2s”的序列循环执行，OLED 同步切换为难过表情。共约 8 秒后自动回到 FOLLOW 模式
- **实地测试：**我们遗憾的发现，我们的小车由于电机驱动能力太差，在摩擦较大的地面上无法完成任何运动，只有在非常光滑的地面上才能完成动作。同时，因为我们的电压转换器不够，无法实现 PWM 通过占空比调速，导致小车在执行转弯时只能依靠一轮往前一轮往后，不太灵敏。

对于此块的测试总体较为顺利，没有遇到大问题，毕竟代码运行没有任何问题，驱动能力太差我们也无法解决，或许换个更强力的电机效果会理想很多。

3.3 OLED 显示功能测试与分析

测试结果：

- **上电初始化：**FPGA 上电后，理想状态，OLED 会完成 SSD1306 初始化序列并显示对应表情，但实际出现花屏，原因是有时候 FPGA 供电电压不稳定。

- **表情切换：**通过语音进行模式切换，理想状态 OLED 会在约 100 ms 内完成重新初始化及新表情显示。但是在实际的验证时多次出现了 OLED 表情没有切换，在每次排查时均发现信号输入正常。我们先尝试修改代码，使用寄存器保持信号的持续输入，但是不稳定状态没有很大改善，最后经过排查，我们认为是四脚 oled 硬件不稳定导致的。

OLED 的不稳定是我们项目调试过程中的一大问题，我们先后尝试了多种方法，包括使用寄存器保持信号、提高 I2C 频率等，但仍然存在偶发的花屏或表情切换失败现象。最终我们认为问题主要出在 OLED 模块本身的硬件质量上，买硬件还是要买好一点的（小车也是这样）。

3.4 实践过程中的优化

3.4.1 语音通信架构方案的迭代

最初设计尝试利用 ESP32 模块连接 Wi-Fi，通过云端调用大语言模型 API 以及网络 ASR/TTS 实现远程智能对话。但是由于硬件问题放弃。

最后改为本地部署 ASRPRO 语音芯片。虽然牺牲了开放式对话能力，但换取了识别正确率与较快的响应速度。

3.4.2 总线通信协议的精简

原计划在 ASRPRO 与主控 FPGA 之间建立标准的 UART 串行异步收发总线。但是过于复杂，且在 FPGA 端需要轮询接收缓冲区，增加了设计复杂度和时序风险。

我们将其精简为引脚电平脉冲采样方案。FPGA 仅需在特定时间窗口内对硬件引脚进行电平锁存即可，降低了数字逻辑的复杂度，提高了硬件抗干扰能力。同时我们也将最初设计的两块 FPGA 和一块 ESP32 精简为仅用两块 FPGA 驱动。

4 项目成果

4.1 实物总览

经过设计、编码、仿真和上板调试，本小组成功实现了一台基于 FPGA 驱动的智能陪伴小车。小车的实物组成部分包括：

- **FPGA 开发板：**作为系统主控制器，搭载 100 MHz 板载晶振，提供全部数字逻辑资源。
- **双路 HC-SR04 超声波传感器：**安装于小车前方左右两侧，实现目标方位感知。

- **双路直流减速电机 + L298N 驱动模块**：提供差速驱动能力，最高 PWM 载波频率 20 kHz。
- **SSD1306 OLED 显示屏**：通过 I2C 总线与 FPGA 连接，实时显示情感表情。
- **电源模块**：由电池为电机驱动供电、搭载两个充电宝为 FPGA 和 ASRPRO 供电。

小车外观如下：

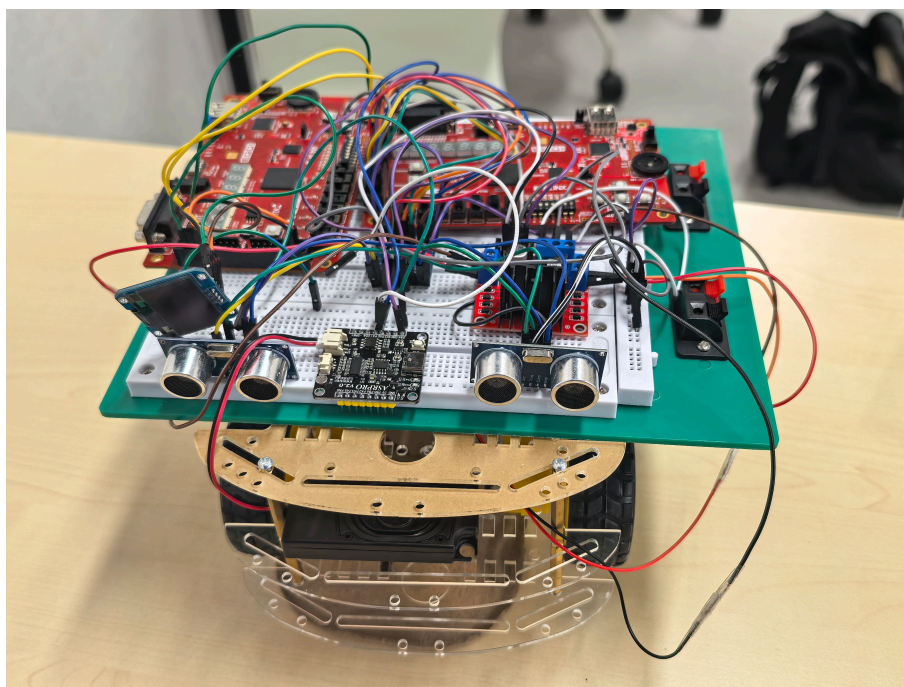


图 7: 小车外观

4.2 实现的功能

- (1) **智能跟随**：通过测距传感器实时采集距离数据，FPGA 快速处理传感信号，输出 PWM 控制信号调节电机转速，实现小车对目标对象的自动稳定跟随。
- (2) **语音情感联动动作功能**：根据不同语音情感识别结果，FPGA 控制小车执行差异化运动动作，实现情感与动作的精准联动，完成拟人化陪伴反馈。
- (3) **OLED 情感表情显示功能**：FPGA 接收语音情感指令，驱动 OLED 显示屏实时显示对应情感表情（笑脸、平静脸、委屈脸），实现用户情感的可视化反馈。

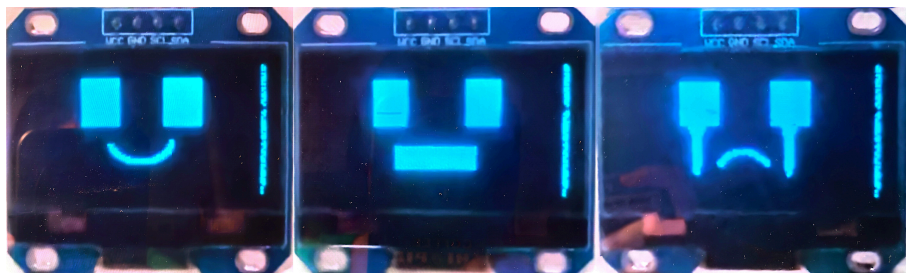


图 8: oled 表情显示

- (4) **语音情感识别功能：**手机/语音采集端通过语音识别算法，采集用户语音信号，提取语音特征并识别情感类型（开心、平静、难过、生气），将情感结果转化为数字控制指令，通过无线通信模块发送至小车 FPGA 主控单元。

我们基本实现了开题答辩中的目标，唯一的改变是因为 ASR 的识别精度问题我们将语音模块从可以识别任意语音的大模型换成了只能识别特定语句的本地部署的识别模块。进一步的，我们完成了差异化动作设计（转圈，前后移动）。

上电启动流程

1. FPGA 加载配置比特流，所有寄存器复位。
2. OLED 上电经历约 500ms 上电延时后，自动发送 SSD1306 初始化命令。
3. 初始化完成后，OLED 显示默认模式对应的表情（平静脸）。
4. 小车默认进入 FOLLOW 模式，超声波测距 FSM 开始循环测量。
5. PID 控制器在每个测量周期结束时更新一次输出，驱动电机完成跟随控制。

跟随模式主循环

1. 触发左侧超声波传感器（ $10\mu s$ 脉冲）。
2. 等待并测量左侧回波时间，存储 left_cnt。
3. 触发右侧超声波传感器（ $10\mu s$ 脉冲）。
4. 等待并测量右侧回波时间，存储 right_cnt。
5. 计算 $avg_cnt = (left_cnt + right_cnt) \gg 2$ 和 $diff_signed = right_cnt - left_cnt$ 。
6. PID 速度环根据 avg_cnt 输出 speed_ctrl；转向环根据 diff_signed 输出 turn_ctrl。

7. 差速混控: $\text{left_duty} = \text{speed_ctrl} + \text{turn_ctrl}$, $\text{right_duty} = \text{speed_ctrl} - \text{turn_ctrl}$ 。
8. 更新 PWM 和 IN1/IN2 方向信号。
9. 等待约 50ms, 进入下一轮测量。

4.3 设计的课程知识运用

4.3.1 运控模块:

1. 运动状态的切换运用了 FSM 的框架, 为了保证小车的基本功能正确运行, 我们设置 happy 状态与 sad 状态执行完成后均无条件进入 neutral 状态, 使小车“始终”以跟随为主要动作。
2. 电机驱动逻辑运用组合逻辑, 在 verilog 中体现为条件分支的行为级描述, 电机接线依据桥式电路原理。

4.3.2 表情控制:

1. FSM 的工程化应用, 顶层采用标准三段式架构, 构建 Idle→Init→Clear→On→Show 的完整工作流程, 严格遵循数字系统“初始化 - 等待 - 执行 - 返回”的经典控制流设计模式;
2. IIC 串行通信协议的实践, 运用 IIC 协议实现 FPGA 与 OLED 显示屏的通讯, 完整实现两线式主从通信、双向三态总线控制、字节传输应答等协议核心机制;
3. 模块化设计思想, 将系统拆分为 IIC 主机、初始化、清屏、显示控制等独立功能模块。

4.4 设计的复杂性

我们的设计在结构和逻辑上具有一定的复杂性, 主要体现在以下几个方面:

- **结构复杂度:** 小车的核心组成是三大模块: 语音模块, 运控模块和表情模块, 三个模块协同工作看似耦合复杂, 但我们通过“情绪标签”的设计规避了复杂的系统设计, 让简单的 2-bit 数据通过组合逻辑得到稳定的电路控制效果。
- **逻辑复杂度:** 由于三个模块的并行机制, 每个模块都采用了分支逻辑, happy, sad, neutral 3 种状态 (算上待机/关机是 4 种) 一一对应, 控制流分支数量较少, 逻辑复杂度低。

- **认知复杂度：**我们的设计思路明确，代码中包括了关键注释，模块所采用的基本都是标准做法（运控模块的控制方式，OLED 的驱动模式）并且特意避开了复杂的状态设计，力图用简单的方式实现核心功能。

4.5 设计特点与创新

- **异构计算架构的协同运行：**不同于传统的单一微处理器方案，我们计划采用两个 FPGA 和语音模块芯片协同的异构架构，他们共用部分信号输入，同时也有各自专控的部分
- **多模态情感映射：**项目不仅停留在“对话”层面，更创新性地引入了语义情感分析。小车可将抽象情绪（喜、怒、哀、乐）实时映射为三位一体的反馈：视觉（OLED 表情）、听觉（语音回复）及物理动作（旋转、靠近、前后运动）。这种“行为语言”极大增强了陪伴的拟人化程度。

5 总结

5.1 设计完成情况

我们小组的项目成功实现了一台基于 FPGA 驱动的智能陪伴小车，完成了全部开题时预定的设计目标：

- (1) **控制算法实现：**设计并实现了可参数化的双环 PID 差速控制算法，能够在理想环境下实现平滑的距离跟随。
- (2) **传感器接口设计：**完成了完整的双路超声波 HC-SR04 测距有限状态机，包含两级寄存器异步信号同步、边沿检测和超时保护等鲁棒性设计，确保系统在传感器异常时不会死锁。
- (3) **通信协议实现：**实现了一个完整的 I2C Master 控制器，成功驱动 SSD1306 OLED 屏幕。
- (4) **人机交互设计：**通过语音实现模式转换，OLED 屏幕实时反馈对应的情感状态，将功能性控制与情感化表达有机结合。
- (5) **电机驱动设计：**输出了完整的 PWM+IN1/IN2 方向信号，适配标准 L298N H 桥驱动模块，支持两路电机的独立正反转。

5.2 不足与展望

我们组的问题主要来自硬件的局限性，具体如下：

- (1) **小车在各种环境跑不了问题：**可以优化小车电机，换功率更大的。
- (2) **可变速度未实现：**因为我们的车电压转换不够，占空比控速未能实现，导致转弯较为不灵敏，和上面一样是硬件优化问题。
- (3) **OLED 显示不稳定问题：**如果换用 7 脚的 oled 和有稳定的电压输入，显示会更稳定。
- (4) **语音识别模块的局限性：**由于 ASRPRO 芯片只能识别预设的命令词，无法实现开放式对话。可以考虑在原来已经实现的云端返回对话内容的基础上提高识别灵敏度，用云端 LLM 模型来实现更自然的语音交互。
- (5) **多模态情感映射的扩展：**目前系统仅支持三种情绪状态（开心、平静、难过），未来可以通过增加更多的情绪类别和对应的动作/表情组合，提升小车的情感表达能力。
- (6) **状态可视化的增加：**在调试 oled 以及运控的过程中我们发现我们很难判断我们的小车目前处于什么状态，未来可以增加一个状态显示模块，引入 FPGA 上的数码管或者 led，用于显示当前小车的模式等信息，方便调试。

5.3 心得

本次基于 FPGA 的智能陪伴小车的项目，我们从最初的架构到最终的实物稳定运行，整个开发过程带给我们诸多超越课本知识的启示与深刻体会。

在项目开发过程中，我们体会到了 Verilog 和 FPGA 的应用。与传统 MCU 的串行轮询不同，FPGA 允许双路测距、PID 算法、I2C 协议栈独立并行运转。这种硬件级别的实时响应，保证了系统较快的联动反馈，感受到了并行设计思维。在各个模块的优化迭代中，我们也深刻理解了模块化设计、状态机设计和通信协议实现的实际应用价值，以及多路径实现的可能性。

无疑，此次课程设计为日后的科研以及工程训练打下坚实的基础。我们不仅掌握了 FPGA 的开发流程，还在实践中锻炼了团队协作、问题分析和解决能力。通过不断调试和优化，我们学会了如何在有限的硬件资源下实现复杂功能，并对系统设计有了更全面的理解。

A 代码结构

A.1 模块结构

系统的完整模块依赖关系如图 9 所示。箭头方向表示“被调用”关系（即上层模块实例化下层模块）。

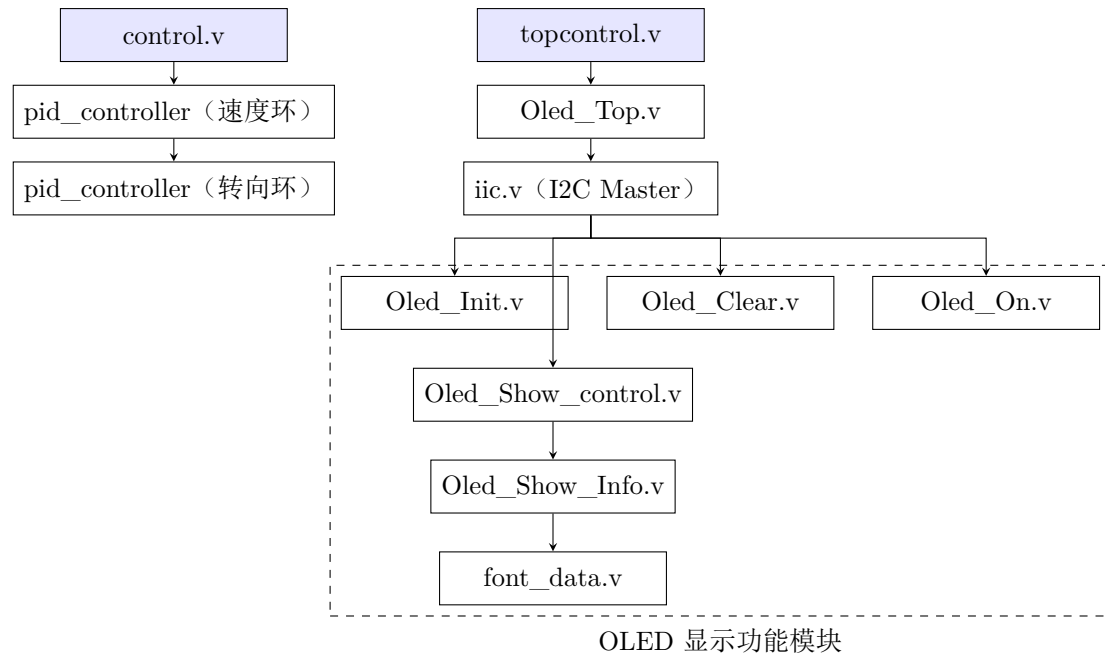


图 9: 系统模块依赖关系图

B 完整代码

B.1 control.v

```

1 module pid_controller #(
2     parameter KP = 16'd256,
3     parameter KI = 16'd128,
4     parameter KD = 16'd64,
5     parameter OUT_MAX = 16'd300,
6     parameter OUT_MIN = -16'd300
7 ) (
8     input wire      clk,
9     input wire      rst,
10    input wire      update,
11    input wire signed [23:0] target,
12    input wire signed [23:0] actual,
13    output reg signed [15:0] output_ctrl
14 );
15
16 reg signed [23:0] integral;
17 reg signed [23:0] last_err;
18 reg signed [31:0] p_term, i_term, d_term;
  
```



```

19 reg signed [24:0] err;
20
21 always @(posedge clk or posedge rst) begin
22     if(rst) begin
23         integral <= 0;
24         last_err <= 0;
25         output_ctrl <= 0;
26     end else if(update) begin
27         err = target - actual;
28         p_term = err * $signed(KP);
29         integral <= integral + err;
30         i_term = integral * $signed(KI);
31         d_term = (err - last_err) * $signed(KD);
32         last_err <= err;
33         output_ctrl <= $signed( (p_term + i_term + d_term) >>> 8
34                                 );
35         if(output_ctrl > OUT_MAX) output_ctrl <= OUT_MAX;
36         if(output_ctrl < OUT_MIN) output_ctrl <= OUT_MIN;
37     end
38 end
39 endmodule
40
41 //
42 // =====
43 //
44 // =====
45
46 module car_follow #(
47     parameter CLK_FREQ      = 100_000_000,
48     parameter DESIRED_CNT   = 294118,
49     parameter PWM_FREQ      = 20000
50 )(
51     input  wire      clk,
52     input  wire      rst,
53     input  wire [1:0] mode,
54     output wire      trig_left,
55     input  wire      echo_left,

```

```

54     output wire      trig_right,
55     input  wire      echo_right,
56     output wire      pwm_left,      // , I wire
57     output reg       in1_left,
58     output reg       in2_left,
59     output wire      pwm_right,     // , I wire
60     output reg       in1_right,
61     output reg       in2_right,
62     output reg       [1:0] mode_code
63 );
64
65 //
-----
66 // 1kHz
67 //
-----

68 reg [16:0] clk_cnt;
69 reg        clk_1kHz;
70 always @(posedge clk or posedge rst) begin
71     if(rst) begin
72         clk_cnt <= 0;
73         clk_1kHz <= 0;
74     end else if(clk_cnt >= CLK_FREQ/1000 - 1) begin
75         clk_cnt <= 0;
76         clk_1kHz <= 1;
77     end else begin
78         clk_cnt <= clk_cnt + 1'b1;
79         clk_1kHz <= 0;
80     end
81 end
82
83 //
-----

84 // 3 2 2 2
85 //
-----

```

```

86 localparam [3:0]
87     IDLE          = 4'd0,
88     TRIG_LEFT     = 4'd1,
89     WAIT_ECHO_L   = 4'd2,
90     MEASURE_L     = 4'd3,
91     STORE_L       = 4'd4,
92     TRIG_RIGHT    = 4'd5,
93     WAIT_ECHO_R   = 4'd6,
94     MEASURE_R     = 4'd7,
95     STORE_R       = 4'd8,
96     WAIT_50MS     = 4'd9;
97
98 reg [3:0] state;
99 //reg [23:0] echo_cnt;
100 //reg [23:0] left_cnt, right_cnt;
101 //reg [11:0] wait_cnt;
102 reg      trig_left_r, trig_right_r;
103 reg      echo_left_ff, echo_left_ff2;
104 reg      echo_right_ff, echo_right_ff2;
105 wire     echo_left_rise, echo_left_fall;
106 wire     echo_right_rise, echo_right_fall;
107
108 always @(posedge clk or posedge rst) begin
109     if(rst) begin
110         echo_left_ff  <= 0;
111         echo_left_ff2 <= 0;
112         echo_right_ff <= 0;
113         echo_right_ff2 <= 0;
114     end else begin
115
116         echo_left_ff  <= echo_left;
117         echo_left_ff2 <= echo_left_ff;
118
119         echo_right_ff <= echo_right;
120         echo_right_ff2 <= echo_right_ff;
121     end
122 end
123 assign echo_left_rise  = echo_left_ff  & ~echo_left_ff2;
124 assign echo_left_fall  = ~echo_left_ff & echo_left_ff2;
125 assign echo_right_rise = echo_right_ff & ~echo_right_ff2;

```

```

126 assign echo_right_fall = ~echo_right_ff & echo_right_ff2;
127 reg [26:0] echo_cnt;          // 23 , I 27
128 reg [26:0] left_cnt, right_cnt;
129 reg [26:0] wait_cnt;         // 12 , I 27
130
131 always @(posedge clk or posedge rst) begin
132     if(rst) begin
133         state <= IDLE;
134         wait_cnt <= 0;
135         echo_cnt <= 0;
136         trig_left_r <= 0;
137         trig_right_r <= 0;
138         left_cnt <= 0;
139         right_cnt <= 0;
140     end else begin
141         case(state)
142             IDLE: begin
143                 trig_left_r <= 0;
144                 trig_right_r <= 0;
145                 state <= TRIG_LEFT;
146             end
147             TRIG_LEFT: begin
148                 if(wait_cnt < 12'd1000) begin
149                     trig_left_r <= 1;
150                     wait_cnt <= wait_cnt + 1;
151                 end else begin
152                     trig_left_r <= 0;
153                     wait_cnt <= 0;
154                     state <= WAIT_ECHO_L;
155                     echo_cnt <= 0;
156                 end
157             end
158             WAIT_ECHO_L: begin
159                 if(echo_left_rise) begin
160                     state <= MEASURE_L;
161                     wait_cnt <= 0;
162                     echo_cnt <= 0;
163                 end else if(wait_cnt >= 12'd3000) begin
164                     state <= TRIG_RIGHT;
165                     wait_cnt <= 0;

```

```
166         end else begin
167             if(clk_1kHz) wait_cnt <= wait_cnt + 1;
168         end
169     end
170 MEASURE_L: begin
171     echo_cnt <= echo_cnt + 1;
172     if(echo_left_fall) begin
173         left_cnt <= echo_cnt;
174         state <= STORE_L;
175         wait_cnt <= 0;
176     end else if(echo_cnt >= 24'd3_000_000) begin
177         state <= TRIG_RIGHT;
178     end
179 end
180 STORE_L: begin
181     state <= TRIG_RIGHT;
182 end
183 TRIG_RIGHT: begin
184     if(wait_cnt < 12'd1000) begin
185         trig_right_r <= 1;
186         wait_cnt <= wait_cnt + 1;
187     end else begin
188         trig_right_r <= 0;
189         wait_cnt <= 0;
190         state <= WAIT_ECHO_R;
191         echo_cnt <= 0;
192     end
193 end
194 WAIT_ECHO_R: begin
195     if(echo_right_rise) begin
196         state <= MEASURE_R;
197         wait_cnt <= 0;
198         echo_cnt <= 0;
199     end else if(wait_cnt >= 12'd3000) begin
200         state <= WAIT_50MS;
201         wait_cnt <= 0;
202     end else begin
203         if(clk_1kHz) wait_cnt <= wait_cnt + 1;
204     end
205 end
```

```

206         MEASURE_R: begin
207             echo_cnt <= echo_cnt + 1;
208             if(echo_right_fall) begin
209                 right_cnt <= echo_cnt;
210                 state <= STORE_R;
211             end else if(echo_cnt >= 24'd3_000_000) begin
212                 state <= WAIT_50MS;
213             end
214         end
215         STORE_R: begin
216             state <= WAIT_50MS;
217         end
218         WAIT_50MS: begin
219             if(wait_cnt >= 12'd50) begin
220                 wait_cnt <= 0;
221                 state <= IDLE;
222             end else if(clk_1kHz) begin
223                 wait_cnt <= wait_cnt + 1;
224             end
225         end
226         default: state <= IDLE;
227     endcase
228 end
229 end
230
231 //      ž      ĩ
232
233 /*
234 //      »      ĩ
235 always @(posedge clk or posedge rst) begin
236     if(rst) begin
237         state <= IDLE;
238         wait_cnt <= 0;
239         echo_cnt <= 0;
240         trig_left_r <= 0;
241         trig_right_r <= 0;
242         left_cnt <= 0;
243         right_cnt <= 0;
244     end else begin
245         case(state)

```

```

246         IDLE: begin
247             trig_left_r <= 0;
248             trig_right_r <= 0;
249             state <= TRIG_LEFT;
250         end
251
252         //  10! s = 1000 ,
253         TRIG_LEFT: begin
254             if(wait_cnt < 27'd1000) begin
255                 trig_left_r <= 1;
256                 wait_cnt <= wait_cnt + 1;
257             end else begin
258                 trig_left_r <= 0;
259                 wait_cnt <= 0;
260                 state <= WAIT_ECHO_L;
261                 echo_cnt <= 0;
262             end
263         end
264
265         //  30ms = 3,000,000 ,      p   clk_1kHz
266         //  30
267         WAIT_ECHO_L: begin
268             if(echo_left_rise) begin
269                 state <= MEASURE_L;
270                 wait_cnt <= 0;
271                 echo_cnt <= 0;
272             end else if(wait_cnt >= 27'd3_000_000) begin
273                 state <= TRIG_RIGHT;
274                 wait_cnt <= 0;
275             end else begin
276                 wait_cnt <= wait_cnt + 1; // p   clk_1kHz
277             end
278         end
279
280         //  30ms = 3,000,000 ,
281         MEASURE_L: begin
282             echo_cnt <= echo_cnt + 1;
283             if(echo_left_fall) begin
284                 left_cnt <= echo_cnt;

```

```

284         state <= STORE_L;
285         wait_cnt <= 0;
286     end else if(echo_cnt >= 27'd3_000_000) begin
287         state <= TRIG_RIGHT;
288     end
289 end
290
291 STORE_L: begin
292     state <= TRIG_RIGHT;
293 end
294
295 //    10! s = 1000 ,
296 TRIG_RIGHT: begin
297     if(wait_cnt < 27'd1000) begin
298         trig_right_r <= 1;
299         wait_cnt <= wait_cnt + 1;
300     end else begin
301         trig_right_r <= 0;
302         wait_cnt <= 0;
303         state <= WAIT_ECHO_R;
304         echo_cnt <= 0;
305     end
306 end
307
308 //    30ms = 3,000,000 ,    p    clk_1kHz
309 //    30ms = 3,000,000 ,    p    clk_1kHz
310 WAIT_ECHO_R: begin
311     if(echo_right_rise) begin
312         state <= MEASURE_R;
313         wait_cnt <= 0;
314         echo_cnt <= 0;
315     end else if(wait_cnt >= 27'd3_000_000) begin
316         state <= WAIT_50MS;
317         wait_cnt <= 0;
318     end else begin
319         wait_cnt <= wait_cnt + 1;    //    p    clk_1kHz
320     end
321 end
end

```



```

322         // 30ms = 3,000,000 ,
323     MEASURE_R: begin
324         echo_cnt <= echo_cnt + 1;
325         if(echo_right_fall) begin
326             right_cnt <= echo_cnt;
327             state <= STORE_R;
328         end else if(echo_cnt >= 27'd3_000_000) begin
329             state <= WAIT_50MS;
330         end
331     end
332
333     STORE_R: begin
334         state <= WAIT_50MS;
335     end
336
337     // 50ms = 5,000,000 , 1μ clk_1kHz 50
338     WAIT_50MS: begin
339         if(wait_cnt >= 27'd5_000_000) begin
340             wait_cnt <= 0;
341             state <= IDLE;
342         end else begin
343             wait_cnt <= wait_cnt + 1; // 1μ clk_1kHz
344         end
345     end
346
347     default: state <= IDLE;
348 endcase
349 end
350 end*/
351 assign trig_left = trig_left_r;
352 assign trig_right = trig_right_r;
353
354 //
355 // 30ms = 3,000,000 , 1μ clk_1kHz 30
356 //

```

```
357 localparam [2:0]
358     MODE_STOP      = 3'd0, //00
359     MODE_FOLLOW    = 3'd1, //01 normal
360     MODE_FWD_REV   = 3'd2, //10 sad
361     MODE_SPIN      = 3'd3; //11 happy
362
363 reg [2:0] current_mode;
364 reg [15:0] timer;
365 reg [1:0] cycle_cnt;
366 reg fwd_rev_state;
367 reg action_busy;
368 reg [1:0] last_mode;
369
370 always @(posedge clk or posedge rst) begin
371     if(rst) begin
372         current_mode <= MODE_FOLLOW;
373         mode_code <= current_mode;
374         timer <= 0;
375         cycle_cnt <= 0;
376         fwd_rev_state <= 0;
377         action_busy <= 0;
378         last_mode <= 2'b11;
379     end else begin
380         last_mode <= mode;
381         if(!action_busy) begin
382             case(mode)
383                 2'b00: begin current_mode <= MODE_STOP;
384                             mode_code <= MODE_STOP; end
385                 2'b01:
386                     if(last_mode != 2'b01) begin
387                         current_mode <= MODE_FWD_REV;
388                         mode_code <= MODE_FWD_REV;
389                         action_busy <= 1;
390                         timer <= 0;
391                         cycle_cnt <= 0;
392                         fwd_rev_state <= 0;
393                     end
394                 2'b10: if(last_mode != 2'b10) begin
395                             current_mode <= MODE_SPIN;
396                             mode_code <= MODE_SPIN;
```

```
397         action_busy <= 1;
398         timer <= 0;
399     end
400     2'b11: begin current_mode <= MODE_FOLLOW;
401             mode_code <= MODE_FOLLOW; end
402     default: begin current_mode <= MODE_FOLLOW;
403             mode_code <= MODE_FOLLOW; end
404 endcase
405 end
406
407 case(current_mode)
408     MODE_FWD_REV: begin
409         if(timer >= 2000) begin
410             timer <= 0;
411             if(fwd_rev_state == 0) fwd_rev_state <= 1;
412             else begin
413                 fwd_rev_state <= 0;
414                 if(cycle_cnt < 2) cycle_cnt <= cycle_cnt
415                     + 1;
416             end
417         end else if(clk_1kHz) timer <= timer + 1;
418
419         if(cycle_cnt == 2 && fwd_rev_state == 1 && timer
420             >= 2000) begin
421             current_mode <= MODE_FOLLOW;
422             action_busy <= 0;
423             cycle_cnt <= 0;
424             fwd_rev_state <= 0;
425         end
426     end
427     MODE_SPIN: begin
428         if(timer >= 10000) begin
429             current_mode <= MODE_FOLLOW;
430             action_busy <= 0;
431             timer <= 0;
432         end else if(clk_1kHz) timer <= timer + 1;
433     end
434     default: ;
435 endcase
```

```

435         if(current_mode != MODE_FWD_REV && current_mode !=
436             MODE_SPIN)
437             action_busy <= 0;
438     end
439 end
440
441 //
442 -----
443 // PID      ž
444 //
445 -----
446
447 reg          pid_update;
448 reg [23:0] avg_cnt;
449 reg signed [15:0] speed_ctrl, turn_ctrl;
450 reg signed [23:0] diff_signed;
451 wire signed [15:0] speed_pid_out, turn_pid_out;
452
453 reg old_state;
454 always @(posedge clk or posedge rst) begin
455     if(rst) begin
456         pid_update <= 0;
457         old_state <= 0;
458     end else begin
459         old_state <= (state == WAIT_50MS);
460         if((state == WAIT_50MS) && !old_state) pid_update <= 1;
461         else pid_update <= 0;
462     end
463 end
464
465 always @(posedge clk or posedge rst) begin
466     if(rst) begin
467         avg_cnt <= 0;
468         diff_signed <= 0;
469     end else begin
470         avg_cnt <= (left_cnt + right_cnt) >> 2;
471         //avg_cnt <= (left_cnt + right_cnt) >> 1;
472         diff_signed <= $signed({1'b0, right_cnt}) - $signed({1'b0, left_cnt})

```

```

        b0, left_cnt});
470     end
471 end
472
473 pid_controller #(.KP(512), .KI(128), .KD(256), .OUT_MAX(200), .
    OUT_MIN(-200)) u_speed_pid (
474     .clk(clk), .rst(rst), .update(pid_update),
475     .target(DESIRED_CNT), .actual(avg_cnt), .output_ctrl(
        speed_pid_out)
476 );
477
478 pid_controller #(.KP(384), .KI(64), .KD(192), .OUT_MAX(150), .
    OUT_MIN(-150)) u_turn_pid (
479     .clk(clk), .rst(rst), .update(pid_update),
480     .target(24'd0), .actual(diff_signed), .output_ctrl(
        turn_pid_out)
481 );
482
483 always @(posedge clk or posedge rst) begin
484     if(rst) begin
485         speed_ctrl <= 0;
486         turn_ctrl <= 0;
487     end else if(pid_update && current_mode == MODE_FOLLOW) begin
488         speed_ctrl <= speed_pid_out;
489         turn_ctrl <= turn_pid_out;
490     end else if(current_mode != MODE_FOLLOW) begin
491         speed_ctrl <= 0;
492         turn_ctrl <= 0;
493     end
494 end
495
496 //
    -----
497 // 计算左右电机的PWM占空比，并赋值给left_duty, right_duty
    -----
498 //
    -----

499 reg signed [8:0] left_duty, right_duty;
500 reg [7:0] pwm_left_val, pwm_right_val;

```

```

501 localparam ACT_SPEED = 8'd150;
502
503 always @(*) begin
504     left_duty = 0;
505     right_duty = 0;
506     case(current_mode)
507         MODE_STOP: ;
508         MODE_FOLLOW: begin
509
510             left_duty = speed_ctrl + turn_ctrl;
511
512             right_duty = speed_ctrl - turn_ctrl;
513         end
514         MODE_FWD_REV: begin
515             if(fwd_rev_state == 0) begin // 正向
516                 left_duty = ACT_SPEED;
517                 right_duty = ACT_SPEED;
518             end else begin // 反向
519                 left_duty = -ACT_SPEED;
520                 right_duty = -ACT_SPEED;
521             end
522         end
523         MODE_SPIN: begin
524             left_duty = ACT_SPEED; // 左轮正向
525             right_duty = -ACT_SPEED; // 右轮反向
526         end
527         default: ;
528     endcase
529 end
530
531 // 将 duty 写入 PWM 寄存器 IN1/IN2 控制电机
532 always @(*) begin
533     pwm_left_val = 0;
534     pwm_right_val = 0;
535     in1_left = 0; in2_left = 0;
536     in1_right = 0; in2_right = 0;
537
538     //
539     if(left_duty > 8'd255) begin
540         pwm_left_val = 8'd255;

```

```
541         in1_left = 1; in2_left = 0;
542     end else if(left_duty < -8'd255) begin
543         pwm_left_val = 8'd255;
544         in1_left = 0; in2_left = 1;
545     end else if(left_duty > 0) begin
546         pwm_left_val = left_duty[7:0];
547         in1_left = 1; in2_left = 0;
548     end else if(left_duty < 0) begin
549         pwm_left_val = -left_duty[7:0];
550         in1_left = 0; in2_left = 1;
551     end else begin
552         pwm_left_val = 0;
553         in1_left = 0; in2_left = 0;
554     end
555
556     //
557     if(right_duty > 8'd255) begin
558         pwm_right_val = 8'd255;
559         in1_right = 1; in2_right = 0;
560     end else if(right_duty < -8'd255) begin
561         pwm_right_val = 8'd255;
562         in1_right = 0; in2_right = 1;
563     end else if(right_duty > 0) begin
564         pwm_right_val = right_duty[7:0];
565         in1_right = 1; in2_right = 0;
566     end else if(right_duty < 0) begin
567         pwm_right_val = -right_duty[7:0];
568         in1_right = 0; in2_right = 1;
569     end else begin
570         pwm_right_val = 0;
571         in1_right = 0; in2_right = 0;
572     end
573 end
574
575 //
576 // PWM      3
577 //
```

```

578 reg [11:0] pwm_cnt;
579 wire [11:0] pwm_period = CLK_FREQ / PWM_FREQ;
580 reg pwm_left_r, pwm_right_r;
581
582 always @(posedge clk or posedge rst) begin
583     if(rst) begin
584         pwm_cnt <= 0;
585         pwm_left_r <= 0;
586         pwm_right_r <= 0;
587     end else begin
588         if(pwm_cnt >= pwm_period - 1)
589             pwm_cnt <= 0;
590         else
591             pwm_cnt <= pwm_cnt + 1;
592         pwm_left_r <= (pwm_cnt < {4'b0, pwm_left_val}) ? 1'b1 :
                    1'b0;
593         pwm_right_r <= (pwm_cnt < {4'b0, pwm_right_val}) ? 1'b1
                    : 1'b0;
594     end
595 end
596
597 assign pwm_left = pwm_left_r;
598 assign pwm_right = pwm_right_r;
599
600 endmodule

```

Listing 1: control.v ——主控模块（PID + 超声波测距 + 电机控制）

B.2 OLED/iic.v

```

1  `timescale 1ns / 1ps
2  //
3  // Company:
4  // Engineer:
5  //
6  // Create Date:      22:40:45 11/20/2017
7  // Design Name:
8  // Module Name:      I2C_Master
9  // Project Name:

```



```

10 // Target Devices:
11 // Tool versions:
12 // Description:
13
14         /*
15         I2C          模块 SCL SDA
16         地址 0x00 SCL, 0x01 SDA - 1
17         1/2 0x00 SCL, 0x01 SDA - 1
18         SDA 0x00 SCL 1
19         模块 1 0x00 SCL 1
20         SCL I200kHz
21         1 0x00 SCL 1 0x01 SCL 1
22         */
23 //
24 // Dependencies:
25 //
26 // Revision:
27 // Revision 0.01 - File Created
28 // Additional Comments:
29 //
30 //
31
32 module I2C_Master(
33     I_Clk_in,
34     I_Rst_n,
35     O_SCL,
36     IO_SDA,
37     //control_sig
38     I_Start, // 1 / 2 0x00
39     O_Done, // 1 / 2 0x01
40     I_R_W_SET, // 0x00 0x01 I1 0x01 IO
41     I_Slave_Addr, // 0x00 1
42     I_R_W_Data, // 0x00 16 I_R_W_Data[15:8] -> reg_addr,
43         I_R_W_Data[7:0] -> W_data, 0x00 0x01 0x02 0x03
44     O_Data, // 0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07
45     O_Error // 0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07
46 );
47 //I/O

```

```

48 input      I_Clk_in;
49 input      I_Rst_n;
50 output     O_SCL;
51 inout      IO_SDA;
52
53 input      I_Start;
54 output     O_Done;
55 input [6:0] I_Slave_Addr;
56 input      I_R_W_SET;
57 input [15:0] I_R_W_Data;
58 output [7:0] O_Data;
59 output     O_Error;
60 /*****      200 kHz SCL 200 kHz SCL
      200 kHz SCL *****/
61 parameter Start_Delay=9'd60; // SDA - 3 1/4 12 ü
      SCL_HIGH2LOW-1 60
62 parameter Stop_Delay=9'd150; // SDA - 3 1/4 12 ü
      SCL_HIGH2LOW-1 150
63 parameter SCL_Period=10'd499; // 150MHz, 100KHz 1500, Clk
      499
64 parameter SCL_LOW_Dest=10'd375; // 3/4, 375, Clk
      13/4, 375, Clk
65 parameter SCL_HIGH2LOW=9'd249; // 1/2, SCL 250,
      Clk
66 parameter ACK_Dect=9'd124; // SCL, 124 ACK 124
67 reg [8:0] R_SCL_Cnt;
68 reg      R_SCL_En;
69
70 assign     O_SCL=(R_SCL_Cnt<=SCL_HIGH2LOW)?1'b1:1'b0; // SCL
71
72 always @ (posedge I_Clk_in or negedge I_Rst_n)
73 begin
74     if (~I_Rst_n)
75     begin
76         R_SCL_Cnt<=9'b0;
77     end
78     else
79     begin
80         if (R_SCL_En)

```

```

81         if (R_SCL_Cnt==SCL_Period)
82             R_SCL_Cnt<=9'b0;
83         else
84             R_SCL_Cnt<=R_SCL_Cnt+9'b1;
85         else
86             R_SCL_Cnt<=9'b0;
87     end
88 end
89
90 /*****SDA 初始化 *****/
91 reg [5:0] R_State;
92 reg R_SDA_I_O_SET; //SDA 输入/输出使能
93 reg R_SDA_t; //SDA 传输时间
94 reg O_Done; //传输完成标志
95 reg [7:0] O_Data; //传输数据
96 reg O_Error; //传输错误标志
97
98 /***** 参数定义 *****/
99 parameter Start=6'd0; //起始地址
100 parameter ReStart=6'd34; //重传地址
101 parameter Stop=6'd56; //结束地址
102
103 always @ (posedge I_Clk_in or negedge I_Rst_n)
104 begin
105     if (~I_Rst_n)
106     begin
107         R_SCL_En<=1'b0; //使能
108         R_State<=6'd0;
109         R_SDA_I_O_SET<=1'b1; //SDA 输入/输出使能
110         R_SDA_t<=1'b1; //SDA 传输时间
111         O_Data<=8'b0;
112         O_Done<=1'b0;
113         O_Error<=1'b0;
114     end
115     else
116     begin
117         if (I_Start) //开始传输
118         begin
119             case(R_State)
120                 Start: //

```

```

121         begin
122             R_SCL_En<=1'b1;
123             O_Error<=1'b0; // 错误标志
124             if (R_SCL_Cnt==Start_Delay)
125                 begin
126                     R_SDA_t<=1'b0; // SCL 为低电平时，SDA 为 0
127                     R_State<=R_State+6'd1;
128                 end
129             else
130                 begin
131                     R_SDA_t<=1'b1;
132                     R_State<=R_State;
133                 end
134             end
135             6'd1,6'd2,6'd3,6'd4,6'd5,6'd6,6'd7: // 7 位地址
136             begin
137                 if (R_SCL_Cnt==SCL_LOW_Dest)
138                     begin
139                         R_SDA_t<=I_Slave_Addr[6'd7-R_State]; // 取出 MSB
140                         // 取出 LSB
141                         R_State<=R_State+6'd1;
142                     end
143                     else
144                         R_State<=R_State;
145                     end
146             6'd8: // 8 位数据
147             begin
148                 if (R_SCL_Cnt==SCL_LOW_Dest)
149                     begin
150                         R_SDA_t<=1'b0;
151                         R_State<=R_State+6'd1;
152                     end
153                     else
154                         R_State<=R_State;
155                     end
156             6'd9: // ACK
157             begin
158                 if (R_SCL_Cnt==SCL_HIGH2LOW) // 8 位数据接收完成，SDA

```

```

158         begin
159             R_SDA_I_O_SET<=1'b0;
160             R_State<=R_State+6'd1;
161         end
162     else
163         R_State<=R_State;
164     end
165 6'd10: // 9, 1 ACK 2 IO 11
        begin
166         if (R_SCL_Cnt==ACK_Dect)
167         begin
168             O_Error<=IO_SDA; //
169             R_State<=R_State+6'd1;
170         end
171         else
172             R_State<=R_State;
173         end
174 6'd11:
        begin
175         if (R_SCL_Cnt==SCL_HIGH2LOW) // 9, 1/2
            SDA 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000
            begin
187         if (R_SCL_Cnt==SCL_LOW_Dest)
188         begin
189             R_SDA_t<=I_R_W_Data[6'd27-R_State]; // MSB-LSB
190             I_R_W_Data[15:8]
191             R_State<=R_State+6'd1;
192         end
193         else

```

```

194         R_State<=R_State;

195     end
196 6'd20: //ACK
197     begin
198         if (R_SCL_Cnt==SCL_HIGH2LOW)// 8, 1/2p
199             SDA
200         begin
201             R_SDA_I_O_SET<=1'b0;
202             R_State<=R_State+6'd1;
203         end
204         else
205             R_State<=R_State;
206         end
207 6'd21: //1/4 ACK
208     begin
209         if (R_SCL_Cnt==ACK_Dect)
210             begin
211                 O_Error<=IO_SDA;//1/4
212                 R_State<=R_State+6'd1;
213             end
214         else
215             R_State<=R_State;
216         end
217 6'd22:
218     begin
219         if (R_SCL_Cnt==SCL_HIGH2LOW) // 9, 1/2p
220             SDA<=1'b0;
221             R_State<=(O_Error)?Stop:((I_R_W_SET)?(R_State
222                 +6'd1):ReStart); //1/4
223             R_SDA_t<=(O_Error|I_R_W_SET)?1'b0:1'b1; //1/4
224             SDA<=I_R_W_SET;
225         end
226         else
227             R_State<=R_State;
228         end
229 6'd23,6'd24,6'd25,6'd26,6'd27,6'd28,6'd29,6'd30:
230     // 8, 1/4

```

```

228         begin
229     if (R_SCL_Cnt==SCL_LOW_Dest)
230         begin
231             R_SDA_t<=I_R_W_Data[6'd30-R_State]; // MSB-
                LSB 8
232             R_State<=R_State+6'd1;
233         end
234     else
235         R_State<=R_State;

                end

236     6'd31: //ACK
237         begin
238     if (R_SCL_Cnt==SCL_HIGH2LOW) // 8, 1/2μ
        SDA
239         begin
240             R_SDA_I_O_SET<=1'b0;
241             R_State<=R_State+6'd1;
242         end
243     else
244         R_State<=R_State;
245     end
246     6'd32: // 1/4 ACK
247     begin
248         if (R_SCL_Cnt==ACK_Dect)
249             begin
250                 O_Error<=IO_SDA; // 1/4
251                 R_State<=R_State+6'd1;
252             end
253         else
254             R_State<=R_State;
255         end
256     6'd33:
257     begin
258         if (R_SCL_Cnt==SCL_HIGH2LOW) // 9, 1/2μ
            SDA 1/2, 1/4
259         begin
260             R_SDA_I_O_SET<=1'b1;
261             R_SDA_t<=1'b0; // 1/2 SDA 1/2

```

```

262         R_State<=Stop; // 1/2 * t
263     end
264     else
265         R_State<=R_State;
266     end
267     ReStart:// 3
268 begin
269     if (R_SCL_Cnt==Start_Delay)
270     begin
271         R_SDA_t<=1'b0; //SCL, -t
272         R_State<=R_State+6'd1;
273     end
274     else
275     begin
276         R_SDA_t<=1'b1;
277         R_State<=R_State;
278     end
279 end
280         6'd35,6'd36,6'd37,6'd38,6'd39,6'
281         d40,6'd41:// * t 7 t
282         begin
283             if (R_SCL_Cnt==SCL_LOW_Dest)
284             begin
285                 R_SDA_t<=I_Slave_Addr[6'd41-R_State]; //
286                 MSB-LSB t
287                 R_State<=R_State+6'd1;
288             end
289             else
290                 R_State<=R_State;
291         end
292         6'd42:// q )1)
293     begin
294         if (R_SCL_Cnt==SCL_LOW_Dest)
295         begin
296             R_SDA_t<=1'b1; // u ±
297             R_State<=R_State+6'd1;
298         end
299         else
300             R_State<=R_State;

```



```

299         end
300     6'd43: //ACK
301         begin
302             if (R_SCL_Cnt==SCL_HIGH2LOW)// 8, 1/2p
303                 SDA
304                 begin
305                     R_SDA_I_O_SET<=1'b0;
306                     R_State<=R_State+6'd1;
307                 end
308             else
309                 R_State<=R_State;
310             end
311         6'd44://ACK
312         begin
313             if (R_SCL_Cnt==ACK_Dect)
314                 begin
315                     O_Error<=IO_SDA;
316                     R_State<=R_State+6'd1;
317                 end
318             else
319                 R_State<=R_State;
320             end
321         6'd45:// 0, 1/2, 3/4, 1 SDA 1/2, 3
322         begin
323             if (R_SCL_Cnt==SCL_HIGH2LOW)// 9, 1/2p
324                 1/2, 3 SDA
325                 begin
326                     R_SDA_I_O_SET<=(O_Error)?1'b1:1'b0;// 1/2, 3
327                     ACK 1/2, 3 SDA 1/2, 3
328                     1 SDA 4 * 1/2
329                     R_State<=(O_Error)?Stop:(R_State+6'd1);
330                     R_SDA_t<=1'b0;
331                 end
332             else
333                 R_State<=R_State;
334             end
335         6'd46,6'd47,6'd48,6'd49,6'd50,6'd51,6'd52,6'd53:

```

```

331         begin
332             if (R_SCL_Cnt==ACK_Dect)
333                 begin
334                     O_Data<={O_Data[6:0],IO_SDA}; // 将 MSB 数据写入 SDA
335                     R_State<=R_State+6'd1;
336                 end
337             else
338                 R_State<=R_State;
339             end
340         6'd54: // 等待 8 个时钟周期，NACK 信号
341         begin
342             if (R_SCL_Cnt==SCL_HIGH2LOW)
343                 begin
344                     R_SDA_I_O_SET<=1'b1; // 将 SDA 拉高
345                     R_SDA_t<=1'b1;
346                     R_State<=R_State+6'd1;
347                 end
348             else
349                 R_State<=R_State;
350             end
351         6'd55: // 等待 9 个时钟周期，SDA 保持高电平
352         begin
353             if (R_SCL_Cnt==SCL_HIGH2LOW)
354                 begin
355                     R_SDA_t<=1'b0;
356                     R_State<=R_State+6'd1;
357                 end
358             else
359                 R_State<=R_State;
360             end
361         Stop: // 结束
362         begin
363             if (R_SCL_Cnt==Stop_Delay)
364                 begin
365                     R_SDA_t<=1'b1;
366                     R_State<=R_State+6'd1;
367                 end

```

```

368         else
369             R_State<=R_State;
370         end
371         6'd57: //          Done žt± 3 1 2
372         begin
373             R_SCL_En<=1'b0;
374             O_Done<=1'b1;//-, Done ž
375             R_State<=R_State+6'd1;
376         end
377         6'd58:
378         begin
379             O_Done<=1'b0;//-µ Done ž
380             R_State<=Start;
381         end
382         default:
383         begin
384             R_SCL_En<=1'b0;//¼
385             R_State<=6'd0;
386             R_SDA_I_O_SET<=1'b1;//İ I
387             R_SDA_t<=1'b1;//SDA İ -,
388             O_Done<=1'b0;
389
390         end
391     endcase
392     end
393 else // ž ž ž» ½³
394     begin
395         R_SCL_En<=1'b0; //¼
396         R_State<=6'd0;
397         R_SDA_I_O_SET<=1'b1;//İ I
398         R_SDA_t<=1'b1; //SDA İ -,
399         O_Done<=1'b0;
400     end
401 end
402
403 /* **** */
404 assign IO_SDA=(R_SDA_I_O_SET)?R_SDA_t:1'bz;
405
406

```

407 `endmodule`

Listing 2: iic.v —— I2C Master 通信协议

B.3 OLED/font_data.v

```
1
2
3
4
5
6 // 字符数据 ?16*16 ? / 顿16个数据，共两 ?
7
8 module font_data(
9
10     input          clk,
11     input          rst,
12
13     input [1:0]     sw,
14     input [5:0]     select, // 字符选择
15     input [2:0]     page_cur, // 当前 ?
16     input [5:0]     index_cur, // 当前 ?
17
18     output [7:0]     data // 数据输出
19 );
20
21 reg [7:0] data1 [31:0];
22 reg [7:0] data2 [31:0];
23 reg [7:0] data3 [31:0];
24 reg [7:0] data4 [31:0];
25 reg [7:0] data5 [31:0];
26 reg [7:0] data6 [31:0];
27 reg [7:0] data7 [31:0];
28 reg [7:0] data8 [31:0];
29 reg [7:0] data9 [31:0];
30 reg [7:0] data10 [31:0];
31 reg [7:0] data11 [31:0];
32 reg [7:0] data12 [31:0];
33
34
```

```

35 assign data = (select == 'd0') ? data1[(index_cur-'d3') + 'd16 *
    page_cur] :
36         (select == 'd1') ? data2[(index_cur-'d3') + 'd16 *
    page_cur] :
37         (select == 'd2') ? data3[(index_cur-'d3') + 'd16 *
    page_cur] :
38         (select == 'd3') ? data4[(index_cur-'d3') + 'd16 *
    page_cur] :
39         (select == 'd4') ? data5[(index_cur-'d3') + 'd16 *
    page_cur] :
40         (select == 'd5') ? data6[(index_cur-'d3') + 'd16 *
    page_cur] :
41         (select == 'd6') ? data7[(index_cur-'d3') + 'd16 *
    page_cur] :
42         (select == 'd7') ? data8[(index_cur-'d3') + 'd16 *
    page_cur] :
43         (select == 'd8') ? data9[(index_cur-'d3') + 'd16 *
    page_cur] :
44         (select == 'd9') ? data10[(index_cur-'d3') + 'd16
    *page_cur] :
45         (select == 'd10') ? data11[(index_cur-'d3') + 'd16
    *page_cur] :
46         (select == 'd11') ? data12[(index_cur-'d3') + 'd16
    *page_cur] : 'd0;
47
48 // data1 (p1, 2 DB)
49 // 12, 32 8
50
51
52 always@(posedge clk or negedge rst)
53 begin
54     if(rst == 1'b0)
55         begin
56             if(sw[0] == 1'b0 && sw[1] == 1'b1) //sad
57                 begin
58                     // data1
59                     data1[0] = 8'hFF; data1[1] = 8'hFF; data1[2] = 8'hFF;
60                     data1[3] = 8'hFF;
61                     data1[4] = 8'hFF; data1[5] = 8'hFF; data1[6] = 8'hFF;
62                     data1[7] = 8'hFF;

```

```
61     data1[8]  = 8'hFF; data1[9]  = 8'hFF; data1[10] = 8'hFF;
        data1[11] = 8'hFF;
62     data1[12] = 8'hFF; data1[13] = 8'hFF; data1[14] = 8'hFF;
        data1[15] = 8'hFF;
63     data1[16] = 8'hFF; data1[17] = 8'hFF; data1[18] = 8'hFF;
        data1[19] = 8'hFF;
64     data1[20] = 8'hFF; data1[21] = 8'hFF; data1[22] = 8'hFF;
        data1[23] = 8'hFF;
65     data1[24] = 8'hFF; data1[25] = 8'hFF; data1[26] = 8'hFF;
        data1[27] = 8'hFF;
66     data1[28] = 8'hFF; data1[29] = 8'hFF; data1[30] = 8'hFF;
        data1[31] = 8'hFF;
67
68     // data2
69     data2[0]  = 8'hFF; data2[1]  = 8'hFF; data2[2]  = 8'hFF;
        data2[3]  = 8'h00;
70     data2[4]  = 8'h00; data2[5]  = 8'h00; data2[6]  = 8'h00;
        data2[7]  = 8'h00;
71     data2[8]  = 8'h00; data2[9]  = 8'h00; data2[10] = 8'h00;
        data2[11] = 8'h00;
72     data2[12] = 8'h00; data2[13] = 8'h00; data2[14] = 8'h00;
        data2[15] = 8'h00;
73     data2[16] = 8'hFF; data2[17] = 8'hFF; data2[18] = 8'hFF;
        data2[19] = 8'h00;
74     data2[20] = 8'h00; data2[21] = 8'h00; data2[22] = 8'h00;
        data2[23] = 8'h00;
75     data2[24] = 8'h00; data2[25] = 8'h00; data2[26] = 8'h00;
        data2[27] = 8'h00;
76     data2[28] = 8'h00; data2[29] = 8'h00; data2[30] = 8'h00;
        data2[31] = 8'h00;
77
78     // data3
79     data3[0]  = 8'h00; data3[1]  = 8'h00; data3[2]  = 8'h00;
        data3[3]  = 8'h00;
80     data3[4]  = 8'h00; data3[5]  = 8'h00; data3[6]  = 8'h00;
        data3[7]  = 8'h00;
81     data3[8]  = 8'h00; data3[9]  = 8'h00; data3[10] = 8'h00;
        data3[11] = 8'h00;
82     data3[12] = 8'h00; data3[13] = 8'hFF; data3[14] = 8'hFF;
        data3[15] = 8'hFF;
```

```
83      data3[16] = 8'h00; data3[17] = 8'h00; data3[18] = 8'h00;
      data3[19] = 8'h00;
84      data3[20] = 8'h00; data3[21] = 8'h00; data3[22] = 8'h00;
      data3[23] = 8'h00;
85      data3[24] = 8'h00; data3[25] = 8'h00; data3[26] = 8'h00;
      data3[27] = 8'h00;
86      data3[28] = 8'h00; data3[29] = 8'hFF; data3[30] = 8'hFF;
      data3[31] = 8'hFF;
87
88      // data4
89      data4[0] = 8'hFF; data4[1] = 8'hFF; data4[2] = 8'hFF;
      data4[3] = 8'hFF;
90      data4[4] = 8'hFF; data4[5] = 8'hFF; data4[6] = 8'hFF;
      data4[7] = 8'hFF;
91      data4[8] = 8'hFF; data4[9] = 8'hFF; data4[10] = 8'hFF;
      data4[11] = 8'hFF;
92      data4[12] = 8'hFF; data4[13] = 8'hFF; data4[14] = 8'hFF;
      data4[15] = 8'hFF;
93      data4[16] = 8'hFF; data4[17] = 8'hFF; data4[18] = 8'hFF;
      data4[19] = 8'hFF;
94      data4[20] = 8'hFF; data4[21] = 8'hFF; data4[22] = 8'hFF;
      data4[23] = 8'hFF;
95      data4[24] = 8'hFF; data4[25] = 8'hFF; data4[26] = 8'hFF;
      data4[27] = 8'hFF;
96      data4[28] = 8'hFF; data4[29] = 8'hFF; data4[30] = 8'hFF;
      data4[31] = 8'hFF;
97
98      // data5
99      data5[0] = 8'h1F; data5[1] = 8'h1F; data5[2] = 8'h1F;
      data5[3] = 8'h1F;
100     data5[4] = 8'h7F; data5[5] = 8'hFF; data5[6] = 8'hFF;
      data5[7] = 8'hFF;
101     data5[8] = 8'hFF; data5[9] = 8'hFF; data5[10] = 8'hFF;
      data5[11] = 8'hFF;
102     data5[12] = 8'h1F; data5[13] = 8'h1F; data5[14] = 8'h1F;
      data5[15] = 8'h1F;
103     data5[16] = 8'h00; data5[17] = 8'h00; data5[18] = 8'h00;
      data5[19] = 8'h00;
104     data5[20] = 8'h00; data5[21] = 8'h01; data5[22] = 8'hFF;
      data5[23] = 8'hFF;
```

```
105     data5[24] = 8'hFF; data5[25] = 8'hFF; data5[26] = 8'h03;
        data5[27] = 8'h00;
106     data5[28] = 8'h00; data5[29] = 8'h00; data5[30] = 8'h00;
        data5[31] = 8'h00;
107
108     // data6
109     data6[0]  = 8'h1F; data6[1]  = 8'h1F; data6[2]  = 8'h1F;
        data6[3]  = 8'h00;
110     data6[4]  = 8'h00; data6[5]  = 8'h00; data6[6]  = 8'h00;
        data6[7]  = 8'h00;
111     data6[8]  = 8'h00; data6[9]  = 8'h00; data6[10] = 8'h00;
        data6[11] = 8'h00;
112     data6[12] = 8'h00; data6[13] = 8'h00; data6[14] = 8'h00;
        data6[15] = 8'h00;
113     data6[16] = 8'h00; data6[17] = 8'h00; data6[18] = 8'h00;
        data6[19] = 8'h00;
114     data6[20] = 8'h00; data6[21] = 8'h00; data6[22] = 8'h00;
        data6[23] = 8'h00;
115     data6[24] = 8'h00; data6[25] = 8'h00; data6[26] = 8'h00;
        data6[27] = 8'h00;
116     data6[28] = 8'h00; data6[29] = 8'h00; data6[30] = 8'h00;
        data6[31] = 8'h00;
117
118     // data7
119     data7[0]  = 8'h00; data7[1]  = 8'h00; data7[2]  = 8'h00;
        data7[3]  = 8'h00;
120     data7[4]  = 8'h00; data7[5]  = 8'h00; data7[6]  = 8'h00;
        data7[7]  = 8'h00;
121     data7[8]  = 8'h00; data7[9]  = 8'h00; data7[10] = 8'h00;
        data7[11] = 8'h00;
122     data7[12] = 8'h00; data7[13] = 8'h1F; data7[14] = 8'h1F;
        data7[15] = 8'h1F;
123     data7[16] = 8'h00; data7[17] = 8'h00; data7[18] = 8'h00;
        data7[19] = 8'h00;
124     data7[20] = 8'h00; data7[21] = 8'h00; data7[22] = 8'h00;
        data7[23] = 8'h00;
125     data7[24] = 8'h00; data7[25] = 8'h00; data7[26] = 8'h00;
        data7[27] = 8'h00;
126     data7[28] = 8'h00; data7[29] = 8'h00; data7[30] = 8'h00;
        data7[31] = 8'h00;
```



```
127
128 // data8
129 data8[0] = 8'h1F; data8[1] = 8'h1F; data8[2] = 8'h3F;
130 data8[3] = 8'hFF;
131 data8[4] = 8'hFF; data8[5] = 8'hFF; data8[6] = 8'hFF;
132 data8[7] = 8'hFF;
133 data8[8] = 8'hFF; data8[9] = 8'h7F; data8[10] = 8'h1F;
134 data8[11] = 8'h1F;
135 data8[12] = 8'h1F; data8[13] = 8'h1F; data8[14] = 8'h1F;
136 data8[15] = 8'h1F;
137 data8[16] = 8'h00; data8[17] = 8'h00; data8[18] = 8'h00;
138 data8[19] = 8'h03;
139 data8[20] = 8'hFF; data8[21] = 8'hFF; data8[22] = 8'hFF;
140 data8[23] = 8'hFF;
141 data8[24] = 8'h01; data8[25] = 8'h00; data8[26] = 8'h00;
142 data8[27] = 8'h00;
143 data8[28] = 8'h00; data8[29] = 8'h00; data8[30] = 8'h00;
144 data8[31] = 8'h00;
145
146 // data9
147 data9[0] = 8'h00; data9[1] = 8'h00; data9[2] = 8'h00;
148 data9[3] = 8'h00;
149 data9[4] = 8'h00; data9[5] = 8'h00; data9[6] = 8'hFF;
150 data9[7] = 8'hFF;
151 data9[8] = 8'hFF; data9[9] = 8'hFF; data9[10] = 8'h00;
152 data9[11] = 8'h00;
153 data9[12] = 8'h00; data9[13] = 8'h00; data9[14] = 8'h00;
154 data9[15] = 8'h00;
155 data9[16] = 8'h00; data9[17] = 8'h00; data9[18] = 8'h00;
156 data9[19] = 8'h00;
157 data9[20] = 8'h00; data9[21] = 8'h00; data9[22] = 8'h0F;
158 data9[23] = 8'h1F;
159 data9[24] = 8'h1F; data9[25] = 8'h01; data9[26] = 8'h00;
160 data9[27] = 8'h00;
161 data9[28] = 8'h00; data9[29] = 8'h00; data9[30] = 8'h00;
162 data9[31] = 8'h00;
163
164 // data10
165 data10[0] = 8'h00; data10[1] = 8'h00; data10[2] = 8'
166 h80; data10[3] = 8'hC0;
```

```
150     data10[4] = 8'hE0; data10[5] = 8'hF0; data10[6] = 8'  
        h38; data10[7] = 8'h3C;  
151     data10[8] = 8'h1C; data10[9] = 8'h0C; data10[10] = 8'  
        h0E; data10[11] = 8'h0E;  
152     data10[12] = 8'h06; data10[13] = 8'h06; data10[14] = 8'  
        h06; data10[15] = 8'h06;  
153     data10[16] = 8'h00; data10[17] = 8'h00; data10[18] = 8'  
        h00; data10[19] = 8'h01;  
154     data10[20] = 8'h01; data10[21] = 8'h00; data10[22] = 8'  
        h00; data10[23] = 8'h00;  
155     data10[24] = 8'h00; data10[25] = 8'h00; data10[26] = 8'  
        h00; data10[27] = 8'h00;  
156     data10[28] = 8'h00; data10[29] = 8'h00; data10[30] = 8'  
        h00; data10[31] = 8'h00;  
  
157  
158     // data11  
159     data11[0] = 8'h06; data11[1] = 8'h06; data11[2] = 8'  
        h0E; data11[3] = 8'h0E;  
160     data11[4] = 8'h0C; data11[5] = 8'h1C; data11[6] = 8'  
        h38; data11[7] = 8'h30;  
161     data11[8] = 8'h70; data11[9] = 8'hE0; data11[10] = 8'  
        hC0; data11[11] = 8'hC0;  
162     data11[12] = 8'h00; data11[13] = 8'h00; data11[14] = 8'  
        h00; data11[15] = 8'h00;  
163     data11[16] = 8'h00; data11[17] = 8'h00; data11[18] = 8'  
        h00; data11[19] = 8'h00;  
164     data11[20] = 8'h00; data11[21] = 8'h00; data11[22] = 8'  
        h00; data11[23] = 8'h00;  
165     data11[24] = 8'h00; data11[25] = 8'h00; data11[26] = 8'  
        h01; data11[27] = 8'h01;  
166     data11[28] = 8'h00; data11[29] = 8'h00; data11[30] = 8'  
        h00; data11[31] = 8'h00;  
  
167  
168     // data12  
169     data12[0] = 8'h00; data12[1] = 8'h00; data12[2] = 8'  
        h00; data12[3] = 8'h00;  
170     data12[4] = 8'hFF; data12[5] = 8'hFF; data12[6] = 8'  
        hFF; data12[7] = 8'hFF;  
171     data12[8] = 8'h00; data12[9] = 8'h00; data12[10] = 8'  
        h00; data12[11] = 8'h00;
```

```

172     data12[12] = 8'h00; data12[13] = 8'h00; data12[14] = 8'
        h00; data12[15] = 8'h00;
173     data12[16] = 8'h00; data12[17] = 8'h00; data12[18] = 8'
        h00; data12[19] = 8'h00;
174     data12[20] = 8'h07; data12[21] = 8'h3F; data12[22] = 8'
        h1F; data12[23] = 8'h03;
175     data12[24] = 8'h00; data12[25] = 8'h00; data12[26] = 8'
        h00; data12[27] = 8'h00;
176     data12[28] = 8'h00; data12[29] = 8'h00; data12[30] = 8'
        h00; data12[31] = 8'h00;

177     end
178     else if (sw[0] == 1'b1 && sw[1] == 1'b0 ) // normal
179     begin
180     data1[0]  = 8'hFF; data1[1]  = 8'hFF; data1[2]  = 8'hFF;
        data1[3]  = 8'hFF;
181         data1[4]  = 8'hFF; data1[5]  = 8'hFF; data1[6]  = 8'
            hFF; data1[7]  = 8'hFF;
182         data1[8]  = 8'hFF; data1[9]  = 8'hFF; data1[10] = 8'
            hFF; data1[11] = 8'hFF;
183         data1[12] = 8'hFF; data1[13] = 8'hFF; data1[14] = 8'
            hFF; data1[15] = 8'hFF;
184         data1[16] = 8'hFF; data1[17] = 8'hFF; data1[18] = 8'
            hFF; data1[19] = 8'hFF;
185         data1[20] = 8'hFF; data1[21] = 8'hFF; data1[22] = 8'
            hFF; data1[23] = 8'hFF;
186         data1[24] = 8'hFF; data1[25] = 8'hFF; data1[26] = 8'
            hFF; data1[27] = 8'hFF;
187         data1[28] = 8'hFF; data1[29] = 8'hFF; data1[30] = 8'
            hFF; data1[31] = 8'hFF;

188
189         // data2
190         data2[0]  = 8'hFF; data2[1]  = 8'h00; data2[2]  = 8'
            h00; data2[3]  = 8'h00;
191         data2[4]  = 8'h00; data2[5]  = 8'h00; data2[6]  = 8'
            h00; data2[7]  = 8'h00;
192         data2[8]  = 8'h00; data2[9]  = 8'h00; data2[10] = 8'
            h00; data2[11] = 8'h00;
193         data2[12] = 8'h00; data2[13] = 8'h00; data2[14] = 8'
            h00; data2[15] = 8'h00;
194         data2[16] = 8'hFF; data2[17] = 8'h00; data2[18] = 8'

```

```
195         h00; data2[19] = 8'h00;
196         data2[20] = 8'h00; data2[21] = 8'h00; data2[22] = 8'
197         h00; data2[23] = 8'h00;
198         data2[24] = 8'h00; data2[25] = 8'h00; data2[26] = 8'
199         h00; data2[27] = 8'h00;
200         data2[28] = 8'h00; data2[29] = 8'h00; data2[30] = 8'
201         h00; data2[31] = 8'h00;
202
203         // data3
204         data3[0] = 8'h00; data3[1] = 8'h00; data3[2] = 8'
205         h00; data3[3] = 8'h00;
206         data3[4] = 8'h00; data3[5] = 8'h00; data3[6] = 8'
207         h00; data3[7] = 8'h00;
208         data3[8] = 8'h00; data3[9] = 8'h00; data3[10] = 8'
209         h00; data3[11] = 8'h00;
210         data3[12] = 8'h00; data3[13] = 8'h00; data3[14] = 8'
211         h00; data3[15] = 8'hFF;
212         data3[16] = 8'h00; data3[17] = 8'h00; data3[18] = 8'
213         h00; data3[19] = 8'h00;
214         data3[20] = 8'h00; data3[21] = 8'h00; data3[22] = 8'
215         h00; data3[23] = 8'h00;
216         data3[24] = 8'h00; data3[25] = 8'h00; data3[26] = 8'
217         h00; data3[27] = 8'h00;
218         data3[28] = 8'h00; data3[29] = 8'h00; data3[30] = 8'
219         h00; data3[31] = 8'hFF;
220
221         // data4
222         data4[0] = 8'hFF; data4[1] = 8'hFF; data4[2] = 8'
223         hFF; data4[3] = 8'hFF;
224         data4[4] = 8'hFF; data4[5] = 8'hFF; data4[6] = 8'
225         hFF; data4[7] = 8'hFF;
226         data4[8] = 8'hFF; data4[9] = 8'hFF; data4[10] = 8'
227         hFF; data4[11] = 8'hFF;
228         data4[12] = 8'hFF; data4[13] = 8'hFF; data4[14] = 8'
229         hFF; data4[15] = 8'hFF;
230         data4[16] = 8'hFF; data4[17] = 8'hFF; data4[18] = 8'
231         hFF; data4[19] = 8'hFF;
232         data4[20] = 8'hFF; data4[21] = 8'hFF; data4[22] = 8'
233         hFF; data4[23] = 8'hFF;
234         data4[24] = 8'hFF; data4[25] = 8'hFF; data4[26] = 8'
```

```
217         hFF; data4[27] = 8'hFF;
218         data4[28] = 8'hFF; data4[29] = 8'hFF; data4[30] = 8'
219         hFF; data4[31] = 8'hFF;
220
221 // data5
222 data5[0] = 8'h3F; data5[1] = 8'h3F; data5[2] = 8'
223 h3F; data5[3] = 8'h3F;
224 data5[4] = 8'h3F; data5[5] = 8'h3F; data5[6] = 8'
225 h3F; data5[7] = 8'h3F;
226 data5[8] = 8'h3F; data5[9] = 8'h3F; data5[10] = 8'
227 h3F; data5[11] = 8'h3F;
228 data5[12] = 8'h3F; data5[13] = 8'h3F; data5[14] = 8'
229 h3F; data5[15] = 8'h3F;
230 data5[16] = 8'h00; data5[17] = 8'h00; data5[18] = 8'
231 h00; data5[19] = 8'h00;
232 data5[20] = 8'h00; data5[21] = 8'h00; data5[22] = 8'
233 h00; data5[23] = 8'h00;
234 data5[24] = 8'h00; data5[25] = 8'h00; data5[26] = 8'
235 h00; data5[27] = 8'h00;
236 data5[28] = 8'h00; data5[29] = 8'h00; data5[30] = 8'
237 h00; data5[31] = 8'h00;
238
239 // data6
240 data6[0] = 8'h3F; data6[1] = 8'h00; data6[2] = 8'
241 h00; data6[3] = 8'h00;
242 data6[4] = 8'h00; data6[5] = 8'h00; data6[6] = 8'
243 h00; data6[7] = 8'h00;
244 data6[8] = 8'h00; data6[9] = 8'h00; data6[10] = 8'
245 h00; data6[11] = 8'h00;
246 data6[12] = 8'h00; data6[13] = 8'h00; data6[14] = 8'
247 h00; data6[15] = 8'h00;
248 data6[16] = 8'h00; data6[17] = 8'h00; data6[18] = 8'
249 h00; data6[19] = 8'h00;
250 data6[20] = 8'h00; data6[21] = 8'h00; data6[22] = 8'
251 h00; data6[23] = 8'h00;
252 data6[24] = 8'h00; data6[25] = 8'h00; data6[26] = 8'
253 h00; data6[27] = 8'h00;
254 data6[28] = 8'h00; data6[29] = 8'h00; data6[30] = 8'
255 h00; data6[31] = 8'h00;
```

```
239 // data7
240 data7[0] = 8'h00; data7[1] = 8'h00; data7[2] = 8'
    h00; data7[3] = 8'h00;
241 data7[4] = 8'h00; data7[5] = 8'h00; data7[6] = 8'
    h00; data7[7] = 8'h00;
242 data7[8] = 8'h00; data7[9] = 8'h00; data7[10] = 8'
    h00; data7[11] = 8'h00;
243 data7[12] = 8'h00; data7[13] = 8'h00; data7[14] = 8'
    h00; data7[15] = 8'h3F;
244 data7[16] = 8'h00; data7[17] = 8'h00; data7[18] = 8'
    h00; data7[19] = 8'h00;
245 data7[20] = 8'h00; data7[21] = 8'h00; data7[22] = 8'
    h00; data7[23] = 8'h00;
246 data7[24] = 8'h00; data7[25] = 8'h00; data7[26] = 8'
    h00; data7[27] = 8'h00;
247 data7[28] = 8'h00; data7[29] = 8'h00; data7[30] = 8'
    h00; data7[31] = 8'h00;
248
249 // data8
250 data8[0] = 8'h3F; data8[1] = 8'h3F; data8[2] = 8'
    h3F; data8[3] = 8'h3F;
251 data8[4] = 8'h3F; data8[5] = 8'h3F; data8[6] = 8'
    h3F; data8[7] = 8'h3F;
252 data8[8] = 8'h3F; data8[9] = 8'h3F; data8[10] = 8'
    h3F; data8[11] = 8'h3F;
253 data8[12] = 8'h3F; data8[13] = 8'h3F; data8[14] = 8'
    h3F; data8[15] = 8'h3F;
254 data8[16] = 8'h00; data8[17] = 8'h00; data8[18] = 8'
    h00; data8[19] = 8'h00;
255 data8[20] = 8'h00; data8[21] = 8'h00; data8[22] = 8'
    h00; data8[23] = 8'h00;
256 data8[24] = 8'h00; data8[25] = 8'h00; data8[26] = 8'
    h00; data8[27] = 8'h00;
257 data8[28] = 8'h00; data8[29] = 8'h00; data8[30] = 8'
    h00; data8[31] = 8'h00;
258
259 // data9
260 data9[0] = 8'h00; data9[1] = 8'h00; data9[2] = 8'
    h00; data9[3] = 8'h00;
261 data9[4] = 8'h00; data9[5] = 8'h00; data9[6] = 8'
```

```
        h00; data9[7]  = 8'h00;
262    data9[8]  = 8'h00; data9[9]  = 8'h00; data9[10] = 8'
        h00; data9[11] = 8'hFF;
263    data9[12] = 8'hFF; data9[13] = 8'hFF; data9[14] = 8'
        hFF; data9[15] = 8'hFF;
264    data9[16] = 8'h00; data9[17] = 8'h00; data9[18] = 8'
        h00; data9[19] = 8'h00;
265    data9[20] = 8'h00; data9[21] = 8'h00; data9[22] = 8'
        h00; data9[23] = 8'h00;
266    data9[24] = 8'h00; data9[25] = 8'h00; data9[26] = 8'
        h00; data9[27] = 8'h07;
267    data9[28] = 8'h07; data9[29] = 8'h07; data9[30] = 8'
        h07; data9[31] = 8'h07;
268
269    // data10
270    data10[0]  = 8'hFF; data10[1]  = 8'hFF; data10[2]  =
        8'hFF; data10[3]  = 8'hFF;
271    data10[4]  = 8'hFF; data10[5]  = 8'hFF; data10[6]  =
        8'hFF; data10[7]  = 8'hFF;
272    data10[8]  = 8'hFF; data10[9]  = 8'hFF; data10[10] =
        8'hFF; data10[11] = 8'hFF;
273    data10[12] = 8'hFF; data10[13] = 8'hFF; data10[14] =
        8'hFF; data10[15] = 8'hFF;
274    data10[16] = 8'h07; data10[17] = 8'h07; data10[18] =
        8'h07; data10[19] = 8'h07;
275    data10[20] = 8'h07; data10[21] = 8'h07; data10[22] =
        8'h07; data10[23] = 8'h07;
276    data10[24] = 8'h07; data10[25] = 8'h07; data10[26] =
        8'h07; data10[27] = 8'h07;
277    data10[28] = 8'h07; data10[29] = 8'h07; data10[30] =
        8'h07; data10[31] = 8'h07;
278
279    // data11
280    data11[0]  = 8'hFF; data11[1]  = 8'hFF; data11[2]  =
        8'hFF; data11[3]  = 8'hFF;
281    data11[4]  = 8'hFF; data11[5]  = 8'hFF; data11[6]  =
        8'hFF; data11[7]  = 8'hFF;
282    data11[8]  = 8'hFF; data11[9]  = 8'hFF; data11[10] =
        8'hFF; data11[11] = 8'hFF;
283    data11[12] = 8'hFF; data11[13] = 8'hFF; data11[14] =
```

```

284         8'hFF; data11[15] = 8'hFF;
285         data11[16] = 8'h07; data11[17] = 8'h07; data11[18] =
286         8'h07; data11[19] = 8'h07;
287         data11[20] = 8'h07; data11[21] = 8'h07; data11[22] =
288         8'h07; data11[23] = 8'h07;
289         data11[24] = 8'h07; data11[25] = 8'h07; data11[26] =
290         8'h07; data11[27] = 8'h07;
291         data11[28] = 8'h07; data11[29] = 8'h07; data11[30] =
292         8'h07; data11[31] = 8'h07;
293
294         // data12
295         data12[0] = 8'hFF; data12[1] = 8'hFF; data12[2] =
296         8'hFF; data12[3] = 8'hFF;
297         data12[4] = 8'h00; data12[5] = 8'h00; data12[6] =
298         8'h00; data12[7] = 8'h00;
299         data12[8] = 8'h00; data12[9] = 8'h00; data12[10] =
300         8'h00; data12[11] = 8'h00;
301         data12[12] = 8'h00; data12[13] = 8'h00; data12[14] =
302         8'h00; data12[15] = 8'h00;
303         data12[16] = 8'h07; data12[17] = 8'h07; data12[18] =
304         8'h07; data12[19] = 8'h07;
305         data12[20] = 8'h00; data12[21] = 8'h00; data12[22] =
306         8'h00; data12[23] = 8'h00;
307         data12[24] = 8'h00; data12[25] = 8'h00; data12[26] =
308         8'h00; data12[27] = 8'h00;
309         data12[28] = 8'h00; data12[29] = 8'h00; data12[30] =
310         8'h00; data12[31] = 8'h00;
311
312         end
313     else if (sw[0] == 1'b1 && sw[1] == 1'b1) // happy
314     begin
315         // data1 (ĩ 1)
316         data1[0] = 8'hFF; data1[1] = 8'hFF; data1[2] = 8'hFF;
317         data1[3] = 8'hFF;
318         data1[4] = 8'hFF; data1[5] = 8'hFF; data1[6] = 8'hFF;
319         data1[7] = 8'hFF;
320         data1[8] = 8'hFF; data1[9] = 8'hFF; data1[10] = 8'hFF;
321         data1[11] = 8'hFF;
322         data1[12] = 8'hFF; data1[13] = 8'hFF; data1[14] = 8'hFF;
323         data1[15] = 8'hFF;
324         data1[16] = 8'hFF; data1[17] = 8'hFF; data1[18] = 8'hFF;

```



```

    data1[19] = 8'hFF;
307 data1[20] = 8'hFF; data1[21] = 8'hFF; data1[22] = 8'hFF;
    data1[23] = 8'hFF;
308 data1[24] = 8'hFF; data1[25] = 8'hFF; data1[26] = 8'hFF;
    data1[27] = 8'hFF;
309 data1[28] = 8'hFF; data1[29] = 8'hFF; data1[30] = 8'hFF;
    data1[31] = 8'hFF;
310
311 // data2 (i    2)
312 data2[0]  = 8'hFF; data2[1]  = 8'hFF; data2[2]  = 8'hFF;
    data2[3]  = 8'hFF;
313 data2[4]  = 8'h00; data2[5]  = 8'h00; data2[6]  = 8'h00;
    data2[7]  = 8'h00;
314 data2[8]  = 8'h00; data2[9]  = 8'h00; data2[10] = 8'h00;
    data2[11] = 8'h00;
315 data2[12] = 8'h00; data2[13] = 8'h00; data2[14] = 8'h00;
    data2[15] = 8'h00;
316 data2[16] = 8'hFF; data2[17] = 8'hFF; data2[18] = 8'hFF;
    data2[19] = 8'hFF;
317 data2[20] = 8'h00; data2[21] = 8'h00; data2[22] = 8'h00;
    data2[23] = 8'h00;
318 data2[24] = 8'h00; data2[25] = 8'h00; data2[26] = 8'h00;
    data2[27] = 8'h00;
319 data2[28] = 8'h00; data2[29] = 8'h00; data2[30] = 8'h00;
    data2[31] = 8'h00;
320
321 // data3 (i    3)
322 data3[0]  = 8'h00; data3[1]  = 8'h00; data3[2]  = 8'h00;
    data3[3]  = 8'h00;
323 data3[4]  = 8'h00; data3[5]  = 8'h00; data3[6]  = 8'h00;
    data3[7]  = 8'h00;
324 data3[8]  = 8'h00; data3[9]  = 8'h00; data3[10] = 8'h00;
    data3[11] = 8'h00;
325 data3[12] = 8'hFF; data3[13] = 8'hFF; data3[14] = 8'hFF;
    data3[15] = 8'hFF;
326 data3[16] = 8'h00; data3[17] = 8'h00; data3[18] = 8'h00;
    data3[19] = 8'h00;
327 data3[20] = 8'h00; data3[21] = 8'h00; data3[22] = 8'h00;
    data3[23] = 8'h00;
328 data3[24] = 8'h00; data3[25] = 8'h00; data3[26] = 8'h00;
```

```

    data3[27] = 8'h00;
329 data3[28] = 8'hFF; data3[29] = 8'hFF; data3[30] = 8'hFF;
    data3[31] = 8'hFF;

330
331 // data4 (ĩ    4)
332 data4[0]  = 8'hFF; data4[1]  = 8'hFF; data4[2]  = 8'hFF;
    data4[3]  = 8'hFF;
333 data4[4]  = 8'hFF; data4[5]  = 8'hFF; data4[6]  = 8'hFF;
    data4[7]  = 8'hFF;
334 data4[8]  = 8'hFF; data4[9]  = 8'hFF; data4[10] = 8'hFF;
    data4[11] = 8'hFF;
335 data4[12] = 8'hFF; data4[13] = 8'hFF; data4[14] = 8'hFF;
    data4[15] = 8'hFF;
336 data4[16] = 8'hFF; data4[17] = 8'hFF; data4[18] = 8'hFF;
    data4[19] = 8'hFF;
337 data4[20] = 8'hFF; data4[21] = 8'hFF; data4[22] = 8'hFF;
    data4[23] = 8'hFF;
338 data4[24] = 8'hFF; data4[25] = 8'hFF; data4[26] = 8'hFF;
    data4[27] = 8'hFF;
339 data4[28] = 8'hFF; data4[29] = 8'hFF; data4[30] = 8'hFF;
    data4[31] = 8'hFF;

340
341 // data5 (ĩ    5)
342 data5[0]  = 8'h7F; data5[1]  = 8'h7F; data5[2]  = 8'h7F;
    data5[3]  = 8'h7F;
343 data5[4]  = 8'h7F; data5[5]  = 8'h7F; data5[6]  = 8'h7F;
    data5[7]  = 8'h7F;
344 data5[8]  = 8'h7F; data5[9]  = 8'h7F; data5[10] = 8'h7F;
    data5[11] = 8'h7F;
345 data5[12] = 8'h7F; data5[13] = 8'h7F; data5[14] = 8'h7F;
    data5[15] = 8'h7F;
346 data5[16] = 8'h00; data5[17] = 8'h00; data5[18] = 8'h00;
    data5[19] = 8'h00;
347 data5[20] = 8'h00; data5[21] = 8'h00; data5[22] = 8'h00;
    data5[23] = 8'h00;
348 data5[24] = 8'h00; data5[25] = 8'h00; data5[26] = 8'h00;
    data5[27] = 8'h00;
349 data5[28] = 8'h00; data5[29] = 8'h00; data5[30] = 8'h00;
    data5[31] = 8'h00;

350
```

```
351 // data6 (i 6)
352 data6[0] = 8'h7F; data6[1] = 8'h7F; data6[2] = 8'h7F;
      data6[3] = 8'h7F;
353 data6[4] = 8'h00; data6[5] = 8'h00; data6[6] = 8'h00;
      data6[7] = 8'h00;
354 data6[8] = 8'h00; data6[9] = 8'h00; data6[10] = 8'h00;
      data6[11] = 8'h00;
355 data6[12] = 8'h00; data6[13] = 8'h00; data6[14] = 8'h00;
      data6[15] = 8'h00;
356 data6[16] = 8'h00; data6[17] = 8'h00; data6[18] = 8'h00;
      data6[19] = 8'h00;
357 data6[20] = 8'h00; data6[21] = 8'h00; data6[22] = 8'h00;
      data6[23] = 8'h00;
358 data6[24] = 8'h00; data6[25] = 8'h00; data6[26] = 8'h00;
      data6[27] = 8'h00;
359 data6[28] = 8'h00; data6[29] = 8'h00; data6[30] = 8'h00;
      data6[31] = 8'h00;
360
361 // data7 (i 7)
362 data7[0] = 8'h00; data7[1] = 8'h00; data7[2] = 8'h00;
      data7[3] = 8'h00;
363 data7[4] = 8'h00; data7[5] = 8'h00; data7[6] = 8'h00;
      data7[7] = 8'h00;
364 data7[8] = 8'h00; data7[9] = 8'h00; data7[10] = 8'h00;
      data7[11] = 8'h00;
365 data7[12] = 8'h7F; data7[13] = 8'h7F; data7[14] = 8'h7F;
      data7[15] = 8'h7F;
366 data7[16] = 8'h00; data7[17] = 8'h00; data7[18] = 8'h00;
      data7[19] = 8'h00;
367 data7[20] = 8'h00; data7[21] = 8'h00; data7[22] = 8'h00;
      data7[23] = 8'h00;
368 data7[24] = 8'h00; data7[25] = 8'h00; data7[26] = 8'h00;
      data7[27] = 8'h00;
369 data7[28] = 8'h00; data7[29] = 8'h00; data7[30] = 8'h00;
      data7[31] = 8'hC0;
370
371 // data8 (i 8)
372 data8[0] = 8'h7F; data8[1] = 8'h7F; data8[2] = 8'h7F;
      data8[3] = 8'h7F;
373 data8[4] = 8'h7F; data8[5] = 8'h7F; data8[6] = 8'h7F;
```

```

    data8[7]  = 8'h7F;
374 data8[8]  = 8'h7F; data8[9]  = 8'h7F; data8[10] = 8'h7F;
    data8[11] = 8'h7F;
375 data8[12] = 8'h7F; data8[13] = 8'h7F; data8[14] = 8'h7F;
    data8[15] = 8'h7F;
376 data8[16] = 8'hC0; data8[17] = 8'h00; data8[18] = 8'h00;
    data8[19] = 8'h00;
377 data8[20] = 8'h00; data8[21] = 8'h00; data8[22] = 8'h00;
    data8[23] = 8'h00;
378 data8[24] = 8'h00; data8[25] = 8'h00; data8[26] = 8'h00;
    data8[27] = 8'h00;
379 data8[28] = 8'h00; data8[29] = 8'h00; data8[30] = 8'h00;
    data8[31] = 8'h00;
380
381 // data9 (ĩ    9)
382 data9[0]  = 8'h00; data9[1]  = 8'h00; data9[2]  = 8'h00;
    data9[3]  = 8'h00;
383 data9[4]  = 8'h00; data9[5]  = 8'h00; data9[6]  = 8'h00;
    data9[7]  = 8'h00;
384 data9[8]  = 8'h00; data9[9]  = 8'h00; data9[10] = 8'h00;
    data9[11] = 8'h00;
385 data9[12] = 8'h00; data9[13] = 8'h00; data9[14] = 8'h01;
    data9[15] = 8'h07;
386 data9[16] = 8'h00; data9[17] = 8'h00; data9[18] = 8'h00;
    data9[19] = 8'h00;
387 data9[20] = 8'h00; data9[21] = 8'h00; data9[22] = 8'h00;
    data9[23] = 8'h00;
388 data9[24] = 8'h00; data9[25] = 8'h00; data9[26] = 8'h00;
    data9[27] = 8'h00;
389 data9[28] = 8'h00; data9[29] = 8'h00; data9[30] = 8'h00;
    data9[31] = 8'h00;
390
391 // data10 (ĩ    10)
392 data10[0] = 8'h0F; data10[1] = 8'h1C; data10[2] = 8'
    h3C; data10[3] = 8'h38;
393 data10[4] = 8'h70; data10[5] = 8'h60; data10[6] = 8'
    hE0; data10[7] = 8'hC0;
394 data10[8] = 8'hC0; data10[9] = 8'hC0; data10[10] = 8'
    h80; data10[11] = 8'h80;
395 data10[12] = 8'h80; data10[13] = 8'h80; data10[14] = 8'
```

```
h80; data10[15] = 8'h80;
396 data10[16] = 8'h00; data10[17] = 8'h00; data10[18] = 8'
    h00; data10[19] = 8'h00;
397 data10[20] = 8'h00; data10[21] = 8'h00; data10[22] = 8'
    h00; data10[23] = 8'h00;
398 data10[24] = 8'h01; data10[25] = 8'h01; data10[26] = 8'
    h01; data10[27] = 8'h01;
399 data10[28] = 8'h01; data10[29] = 8'h01; data10[30] = 8'
    h01; data10[31] = 8'h01;
400
401 // data11 (ĩ      11)
402 data11[0]  = 8'h80; data11[1]  = 8'h80; data11[2]  = 8'
    h80; data11[3]  = 8'h80;
403 data11[4]  = 8'h80; data11[5]  = 8'h80; data11[6]  = 8'
    hC0; data11[7]  = 8'hC0;
404 data11[8]  = 8'hE0; data11[9]  = 8'hE0; data11[10] = 8'
    h70; data11[11] = 8'h78;
405 data11[12] = 8'h3C; data11[13] = 8'h1E; data11[14] = 8'
    h0F; data11[15] = 8'h0F;
406 data11[16] = 8'h01; data11[17] = 8'h01; data11[18] = 8'
    h01; data11[19] = 8'h01;
407 data11[20] = 8'h01; data11[21] = 8'h01; data11[22] = 8'
    h01; data11[23] = 8'h01;
408 data11[24] = 8'h00; data11[25] = 8'h00; data11[26] = 8'
    h00; data11[27] = 8'h00;
409 data11[28] = 8'h00; data11[29] = 8'h00; data11[30] = 8'
    h00; data11[31] = 8'h00;
410
411 // data12 (ĩ      12)
412 data12[0]  = 8'h03; data12[1]  = 8'h00; data12[2]  = 8'
    h00; data12[3]  = 8'h00;
413 data12[4]  = 8'h00; data12[5]  = 8'h00; data12[6]  = 8'
    h00; data12[7]  = 8'h00;
414 data12[8]  = 8'h00; data12[9]  = 8'h00; data12[10] = 8'
    h00; data12[11] = 8'h00;
415 data12[12] = 8'h00; data12[13] = 8'h00; data12[14] = 8'
    h00; data12[15] = 8'h00;
416 data12[16] = 8'h00; data12[17] = 8'h00; data12[18] = 8'
    h00; data12[19] = 8'h00;
417 data12[20] = 8'h00; data12[21] = 8'h00; data12[22] = 8'
```

```

418         h00; data12[23] = 8'h00;
        data12[24] = 8'h00; data12[25] = 8'h00; data12[26] = 8'
419         h00; data12[27] = 8'h00;
        data12[28] = 8'h00; data12[29] = 8'h00; data12[30] = 8'
420         h00; data12[31] = 8'h00;
        end
421     end
422 end
423 endmodule

```

Listing 3: font_data.v ——16×16 表情字模数据

B.4 OLED/Oled_Top.v

```

1 //oled顶层模块
2 module    Oled_Top(
3     input        clk,
4     input        rst,
5     input[1:0]    sw,
6     input        clear_key,
7     input        on_key,
8     output        vcc,
9
10    output        scl,
11    inout         sda
12 );
13 localparam      RST_T          =    1'b0;          //复
14    位有效
15 localparam      Oled_Idle      =    11'b000_0000_0001;    //
16    oled初始?
17 localparam      Oled_Init      =    11'b000_0000_0010;    //
18    oled初始?
19 localparam      Oled_Clear     =    11'b000_0000_0100;    //
20    oled初始?
21 localparam      Oled_On        =    11'b000_0000_1000;    //
22    oled初始?
23 localparam      Oled_Show      =    11'b000_0001_0000;    //
24    oled初始?

```

```
21
22 reg[10:0]      state    ,    next_state;
23 //init
24 wire          init_req;          //初始化请 ?
25 wire          init_finish;
26 wire[23:0]    init_data;
27 reg[35:0]     delay_time;
28
29 //clear
30 wire          clear_req;          //初始化请 ?
31 wire          clear_ack;
32 wire[23:0]    clear_data;
33
34 //on
35 wire          on_req;             //初始化请 ?
36 wire          on_ack;
37 wire[23:0]    on_data;
38
39 //show
40 wire[6:0]     start_x;
41 wire[3:0]     start_y;
42 wire[5:0]     show_select;
43 wire          show_req;
44 wire          show_ack;
45 wire[23:0]    show_data;
46 wire          show_end;
47
48 wire          w_ack;              //IIC传输应答
49 reg[23:0]     iic_data;
50 wire          iic_req;
51
52 assign vcc = 1'b1;
53
54
55 always@(*)
56 begin
57     case(state)
58         Oled_Init:
59             iic_data <=init_data;
60         Oled_Idle:
```

```
61         iic_data <=23'd0;
62     Oled_Clear:
63         iic_data <=clear_data;
64     Oled_On:
65         iic_data <=on_data;
66     Oled_Show:
67         iic_data <=show_data;
68     default:
69         iic_data <=23'd0;
70     endcase
71 end
72
73 assign iic_req = ( init_req == 1'b1 || clear_req == 1'b1 ||
74     on_req == 1'b1 || show_req == 1'b1)? 1'b1 : 1'b0;
75
76
77 assign clear_req = (state == Oled_Clear) ? 1'b1 : 1'b0;
78 assign on_req = (state == Oled_On) ? 1'b1 : 1'b0;
79 assign show_req = (state == Oled_Show) ? 1'b1 : 1'b0;
80 assign init_req = (state == Oled_Init && delay_time >='d500_000)
81     ? 1'b1 : 1'b0;
82
83 //上电延时
84 always@(posedge clk or negedge rst)
85 begin
86     if(rst == RST_T)
87         delay_time <= 'd0;
88     else if(delay_time >= 'd500_000)
89         delay_time <= delay_time;
90     else
91         delay_time <= delay_time + 1'b1;
92 end
93
94 always@(posedge clk or negedge rst)
95 begin
96     if(rst == RST_T)
97         state <= Oled_Init;
98     else
99         state <= next_state;
```



```
99  end
100
101  always@(*)
102  begin
103      case(state)
104          Oled_Init:
105              if(init_finish == 1'b1)
106                  next_state <= Oled_Idle;
107              else
108                  next_state <= Oled_Init;
109          Oled_Idle:
110              if(clear_key == 1'b0)
111                  next_state <= Oled_Clear;
112              else if(on_key == 1'b0)
113                  next_state <= Oled_Show;
114              else
115                  next_state <= Oled_Idle;
116          Oled_Clear:
117              if(clear_ack == 1'b1)
118                  next_state <= Oled_Idle;
119              else
120                  next_state <= Oled_Clear;
121          Oled_On:
122              if(on_ack == 1'b1)
123                  next_state <= Oled_Idle;
124              else
125                  next_state <= Oled_On;
126          Oled_Show:
127              if(show_end == 1'b1)
128                  next_state <= Oled_Idle;
129              else
130                  next_state <= Oled_Show;
131          default:
132              next_state <= Oled_Idle;
133      endcase
134  end
135
136
137
138
```

```

139
140 I2C_Master I2C_Master_V(
141     .I_Clk_in      (clk),
142     .I_Rst_n       (rst),
143     .O_SCL         (scl),
144     .IO_SDA        (sda),
145     //control_sig
146     .I_Start       (iic_req),
147     .O_Done        (w_ack),
148     .I_R_W_SET     (1),
149     .I_Slave_Addr  (iic_data[23:17]),
150     .I_R_W_Data    (iic_data[15:0]),
151     .O_Data        (),
152     .O_Error       ()
153 );
154
155
156 Oled_Init      Oled_Init_HP(
157
158     .clk           (clk),
159     .rst           (rst),
160     .init_req      (init_req),      //初始化请 ?
161     .write_done    (w_ack),         // ?组初始化数据完成信号
162     .init_finish   (init_finish),   //初始化完成输 ?
163     .Init_data     (init_data)      //初始化的数据
164 );
165
166
167 Oled_Clear Oled_Clear_HP(
168     .clk           (clk),
169     .rst           (rst),
170
171     .write_done    (w_ack),         //清除 ?组数据完 ?
172
173     .clear_req     (clear_req),     //清除请求
174     .clear_ack     (clear_ack),     //清除完成
175
176     .clear_data    (clear_data)     //清除数据的命 ?
177 );
178

```

```

179 Oled_On  Oled_On_HP(
180     .clk      (clk),
181     .rst      (rst),
182     .write_done (w_ack),          //写入 ?组数据完 ?
183     .On_req    (on_req),          // ?启请 ?
184     .On_ack    (on_ack),          // ?启完 ?
185     .On_data   (on_data)          // ?启数据的命令
186 );
187
188 Oled_Show_control  Oled_Show_control_HP(
189
190     .clk      (clk),
191     .rst      (rst),
192     .show_ack  (show_ack),
193     .show_req  (show_req),
194     .show_end  (show_end),
195     .start_x   (start_x),
196     .start_y   (start_y),
197     .show_select(show_select)      //显示数据索引
198 );
199
200 Oled_Show_Info  Oled_Show_Info_HP(
201     .clk      (clk),
202     .rst      (rst),
203     .sw       (sw),
204     .write_done (w_ack),          //清除 ?组数据完 ?
205
206     //起始坐标
207     .start_x   (start_x),
208     .start_y   (start_y),
209
210     .show_select(show_select),      //显示数据索引
211
212     .show_req   (show_req),          //显示请求
213     .show_ack   (show_ack),          //显示完成
214
215     .show_data  (show_data)          //清除数据的命 ?
216 );
217
218 endmodule

```

Listing 4: Oled_Top.v ——OLED 顶层调度状态机

B.5 OLED/Oled_Init.v

```
1
2
3
4
5
6 //oled_init 初始化模 ?
7 module Oled_Init(
8
9     input          clk,
10    input          rst,
11
12    input          init_req,
13    input          write_done,
14
15    output         init_finish,
16
17    output [23:0]   Init_data
18 );
19
20 localparam       RST_T          =    1'b0;
21
22 reg [23:0]        Init_data_reg;
23 reg [4:0]         Init_index;
24
25
26 assign Init_data  = Init_data_reg;
27 assign init_finish = (Init_index >= 'd26 && write_done == 1'b1)
    ? 1'b1 : 1'b0;
28
29
30
31 always@(posedge clk or negedge rst)
32 begin
33     if(rst == RST_T)
```

```

34     Init_index <= 'd0;
35     else if(Init_index == 'd26 && write_done == 1'b1 )
36         Init_index <= 'd0;
37     else if(write_done == 1'b1 && init_req == 1'b1)
38         Init_index <= Init_index + 1'b1;
39     else
40         Init_index <= Init_index;
41 end
42
43
44 //always@(posedge clk or negedge rst)
45 //begin
46 //    if(rst == RST_T)
47 //        Init_finish_reg <= 1'b0;
48 //    else if(Init_index >= 'd26 && write_done == 1'b1)
49 //        Init_finish_reg <= 1'b1;
50 //    else
51 //        Init_finish_reg <= 1'b0;
52 //end
53
54 always@(*)
55 begin
56     case(Init_index)
57         'd0:    Init_data_reg <= {8'h78,8'h00,8'hAE};
58         'd1:    Init_data_reg <= {8'h78,8'h00,8'h00};
59         'd2:    Init_data_reg <= {8'h78,8'h00,8'h10};
60         'd3:    Init_data_reg <= {8'h78,8'h00,8'h40};
61         'd4:    Init_data_reg <= {8'h78,8'h00,8'hB0};
62         'd5:    Init_data_reg <= {8'h78,8'h00,8'h81};
63         'd6:    Init_data_reg <= {8'h78,8'h00,8'hFF};
64         'd7:    Init_data_reg <= {8'h78,8'h00,8'hA1};
65         'd8:    Init_data_reg <= {8'h78,8'h00,8'hA6};
66         'd9:    Init_data_reg <= {8'h78,8'h00,8'hA8};
67         'd10:   Init_data_reg <= {8'h78,8'h00,8'h3F};
68         'd11:   Init_data_reg <= {8'h78,8'h00,8'hC8};
69         'd12:   Init_data_reg <= {8'h78,8'h00,8'hD3};
70         'd13:   Init_data_reg <= {8'h78,8'h00,8'h00};
71         'd14:   Init_data_reg <= {8'h78,8'h00,8'hD5};
72         'd15:   Init_data_reg <= {8'h78,8'h00,8'h80};
73         'd16:   Init_data_reg <= {8'h78,8'h00,8'hD8};

```

```

74      'd17:    Init_data_reg <= {8'h78,8'h00,8'h05};
75      'd18:    Init_data_reg <= {8'h78,8'h00,8'hD9};
76      'd19:    Init_data_reg <= {8'h78,8'h00,8'hF1};
77      'd20:    Init_data_reg <= {8'h78,8'h00,8'hDA};
78      'd21:    Init_data_reg <= {8'h78,8'h00,8'h12};
79      'd22:    Init_data_reg <= {8'h78,8'h00,8'hDB};
80      'd23:    Init_data_reg <= {8'h78,8'h00,8'h30};
81      'd24:    Init_data_reg <= {8'h78,8'h00,8'h8D};
82      'd25:    Init_data_reg <= {8'h78,8'h00,8'h14};
83      'd26:    Init_data_reg <= {8'h78,8'h00,8'hAF};
84      default:
85          Init_data_reg <= {8'h78,8'h00,8'hAE};
86      endcase
87  end
88
89
90
91
92
93
94
95
96  endmodule

```

Listing 5: Oled_Init.v ——SSD1306 初始化配置

B.6 OLED/Oled_Show_control.v

```

1  // 显示控制
2  module Oled_Show_control(
3
4      input          clk,
5      input          rst,
6      input          show_ack,
7      input          show_req,
8      output         show_end,
9      output [6:0]   start_x,
10     output [3:0]   start_y,
11     output [5:0]   show_select // 显示内容
12 );

```

```

13
14 //===== i3/4 楼 =====
15 localparam STR_ONE_X_STR =7'd32; //p 1 p x
16 localparam STR_ONE_Y_STR =4'd1; //p 1 p y
17 localparam STR_TWO_X_STR =7'd32; //p 2 p x
18 localparam STR_TWO_Y_STR =4'd3; //p 2 p y
19 localparam STR_THREE_X_STR =7'd32; //p 3 p x
20 localparam STR_THREE_Y_STR =4'd5; //p 3 p y
21 localparam STR_TO_STR =7'd0; // 1/4
22 localparam STR_Y_TO_STR_Y =8'd0; //
23 localparam STR_WIDTH =7'd16; // i
24 localparam STR_NUM =6'd12; // 3
25 localparam STR_TWO_INDEX =6'd3; //p 2
26 localparam STR_THREE_INDEX =6'd7; //p 3
27
28 //===== 1/4 J =====
29 reg[6:0] start_x_reg;
30 reg[3:0] start_y_reg;
31 reg[5:0] show_select_reg;
32
33 //=====
34 assign start_x = start_x_reg;
35 assign start_y = start_y_reg;
36 assign show_select = show_select_reg;
37
38 // 0
39 assign show_end = (show_ack == 1'b1 && show_select_reg ==
STR_NUM) ? 1'b1 : 1'b0;
40
41 //===== 1/4 =====
42 always@(posedge clk or negedge rst)

```

```

43 begin
44     if(rst == 1'b0)
45         show_select_reg <= 'd0;
46     else if(show_req == 1'b1 && show_ack == 1'b1)
47         show_select_reg <= show_select_reg + 1'b1;
48     else if(show_req == 1'b1)
49         show_select_reg <= show_select_reg;
50     else
51         show_select_reg <= 'd0;
52 end
53
54 //=====  ±      °  3      2
55
56 always@(posedge clk or negedge rst)
57 begin
58     if(rst == 1'b0)
59         begin
60             // 1
61             start_x_reg <= STR_ONE_X_STR;
62             start_y_reg <= STR_ONE_Y_STR;
63         end
64     else if(show_req == 1'b1 && show_ack == 1'b1)
65         begin
66             if(show_select_reg == STR_TWO_INDEX) // 2
67                 begin
68                     start_x_reg <= STR_TWO_X_STR;
69                     start_y_reg <= STR_TWO_Y_STR;
70                 end
71             else if(show_select_reg == STR_THREE_INDEX) // 3
72                 begin
73                     start_x_reg <= STR_THREE_X_STR;
74                     start_y_reg <= STR_THREE_Y_STR;
75                 end
76             else //
77                 begin
78                     start_x_reg <= start_x_reg + STR_WIDTH + STR_TO_STR;
79                     start_y_reg <= start_y_reg;
80                 end
81         end
82     end
83 end

```



```

81     else if(show_req == 1'b1)
82     begin
83         start_x_reg <= start_x_reg;
84         start_y_reg <= start_y_reg;
85     end
86     else begin
87         // 1/2 μs
88         start_x_reg <= STR_ONE_X_STR;
89         start_y_reg <= STR_ONE_Y_STR;
90     end
91 end
92
93 endmodule

```

Listing 6: Oled_Show_control.v ——显示坐标定位

B.7 OLED/Oled_Show_Info.v

```

1
2 //显示信息 16*16
3 module Oled_Show_Info(
4     input        clk,
5     input        rst,
6     input [1:0]   sw,
7     input        write_done,    //清除 ？组数据完 ？
8
9     //起始坐标
10    input [6:0]    start_x,
11    input [3:0]    start_y,
12
13    input [5:0]    show_select,    //显示数据索引
14
15    input        show_req,    //显示请求
16    output       show_ack,    //显示完成
17
18    output [23:0]  show_data    //清除数据的命 ？
19 );
20
21 localparam      RST_T        =    1'b0;    //复
    位有效

```

```
22 localparam          STR_WIDTH      =6'd16; // 字符宽大 ?
23 localparam          SHOW_INDEX_SET =(STR_WIDTH+3)-1; //6'd18
24 localparam          SHOW_PAGE_SET  =3'd1; //16/2-1=1
25
26 reg[5:0]            show_index;
27 reg[2:0]            show_page;
28 reg[23:0]           show_data_reg;
29
30 wire[7:0]           data;
31
32
33 assign show_data = show_data_reg;
34 assign show_ack = (show_index >= SHOW_INDEX_SET && show_page >=
    SHOW_PAGE_SET && write_done == 1'b1) ? 1'b1 : 1'b0; // 完成 ? 个
    中文字符写 ?
35
36 always@(posedge clk or negedge rst)
37 begin
38     if(rst == RST_T)
39         show_index <= 'd0;
40     else if(show_index == SHOW_INDEX_SET && write_done == 1'b1
41         )
42         show_index <= 'd0;
43     else if(write_done == 1'b1 && show_req == 1'b1)
44         show_index <= show_index + 1'b1;
45     else
46         show_index <= show_index;
47 end
48 // 设置 ?
49 always@(posedge clk or negedge rst)
50 begin
51     if(rst == RST_T)
52         show_page <= 'd0;
53     else if(show_index == SHOW_INDEX_SET && write_done == 1'b1
54         && show_page == SHOW_PAGE_SET)
55         show_page <= 'd0;
56     else if(show_index == SHOW_INDEX_SET && write_done == 1'b1)
57         show_page <= show_page + 1'b1;
58     else
59         show_page <= show_page;
```

```

58 end
59
60 always@(*)
61 begin
62     case(show_index)
63         'd0:show_data_reg <= {8'h78,8'h00,8'hb0 + show_page+
            start_y};
64         'd1:show_data_reg <= {8'h78,8'h00,8'h00 +start_x[3:0]};
65         'd2:show_data_reg <= {8'h78,8'h00,8'h10 + start_x[6:4]};
66         default:show_data_reg <= {8'h78,8'h40, data};
67     endcase
68 end
69
70
71 font_data font_data_HP(
72
73     .clk      (clk),
74     .rst      (rst),
75
76     .sw       (sw),
77     .select   (show_select), //字符选择
78     .page_cur (show_page),   //当前 ?
79     .index_cur(show_index),   //当前 ?
80
81     .data      (data)         //数据输出
82 );
83
84
85 endmodule

```

Listing 7: Oled_Show_Info.v ——显示信息输出

B.8 OLED/Oled_Clear.v

```

1
2
3
4
5
6

```

```
7
8
9
10
11 module Oled_Clear(
12     input      clk,
13     input      rst,
14
15     input      write_done,      //清除 ? 组数据完 ?
16
17     input      clear_req,
18     output     clear_ack,
19
20     output [23:0] clear_data
21 );
22
23 localparam      RST_T          =    1'b0;
24
25 reg [23:0]       clear_data_reg;
26
27
28 reg [3:0]        clear_page;
29 reg [7:0]        clear_index;
30
31 assign clear_data = clear_data_reg;
32 assign clear_ack = (clear_index >= 'd130 && clear_page >= 'd7 &&
33     write_done == 1'b1) ? 1'b1 : 1'b0;
34
35 always@(posedge clk or negedge rst)
36 begin
37     if(rst == RST_T)
38         clear_index <= 'd0;
39     else if(clear_index == 'd130 && write_done == 1'b1 )
40         clear_index <= 'd0;
41     else if(write_done == 1'b1 && clear_req == 1'b1)
42         clear_index <= clear_index + 1'b1;
43     else
44         clear_index <= clear_index;
45 end
```

```

46
47 always@(posedge clk or negedge rst)
48 begin
49     if(rst == RST_T)
50         clear_page <= 'd0;
51     else if(clear_index == 'd130 && write_done == 1'b1 &&
52         clear_page == 'd7)
53         clear_page <= 'd0;
54     else if(clear_index == 'd130 && write_done == 1'b1)
55         clear_page <= clear_page + 1'b1;
56     else
57         clear_page <= clear_page;
58 end
59 always@(*)
60 begin
61     case(clear_index)
62         'd0:clear_data_reg <= {8'h78,8'h00,8'h0 + clear_page};
63         'd1:clear_data_reg <= {8'h78,8'h00,8'h00};
64         'd2:clear_data_reg <= {8'h78,8'h00,8'h10};
65         default:clear_data_reg <= {8'h78,8'h40,8'h00};
66     endcase
67 end
68
69
70
71
72 endmodule

```

Listing 8: Oled_Clear.v ——全屏清除控制

B.9 OLED/Oled_On.v

```

1
2
3
4
5
6
7 module Oled_On(

```

```
8      input      clk,
9      input      rst,
10
11     input      write_done,
12
13     input      On_req,
14     output     On_ack,
15
16     output [23:0] On_data
17 );
18
19 localparam      RST_T          =    1'b0;
20
21 reg [23:0]      On_data_reg;
22
23
24 reg [3:0]      On_page;
25 reg [7:0]      On_index;
26
27 assign On_data  = On_data_reg;
28 assign On_ack = (On_index >= 'd130 && On_page >= 'd7 &&
29     write_done == 1'b1) ? 1'b1 : 1'b0;
30
31 always@(posedge clk or negedge rst)
32 begin
33     if(rst == RST_T)
34         On_index <= 'd0;
35     else if(On_index == 'd130 && write_done == 1'b1 )
36         On_index <= 'd0;
37     else if(write_done == 1'b1 && On_req == 1'b1)
38         On_index <= On_index + 1'b1;
39     else
40         On_index <= On_index;
41 end
42 // 设置 ?
43 always@(posedge clk or negedge rst)
44 begin
45     if(rst == RST_T)
46         On_page <= 'd0;
```

```

47     else if(On_index == 'd130 && write_done == 1'b1 && On_page
        == 'd7)
48         On_page <= 'd0;
49     else if(On_index == 'd130 && write_done == 1'b1)
50         On_page <= On_page + 1'b1;
51     else
52         On_page <= On_page;
53 end
54 always@(*)
55 begin
56     case(On_index)
57         'd0: On_data_reg <= {8'h78, 8'h00, 8'hb0 + On_page};
58         'd1: On_data_reg <= {8'h78, 8'h00, 8'h00};
59         'd2: On_data_reg <= {8'h78, 8'h00, 8'h10};
60         default: On_data_reg <= {8'h78, 8'h40, 8'hFF};
61     endcase
62
63 end
64
65
66 endmodule

```

Listing 9: Oled_On.v——全屏点亮控制

B.10 OLED/topcontrol.v

```

1 module Oled_Top_Wrapper(
2     input          clk,
3     input          rst_n,          // 复位信号
4     input  [1:0]   sw,             // 2位开关
5     output         vcc,
6     output         scl,
7     inout          sda
8 );
9
10 // ===== SW 2位开关 + 复位信号 =====
11 reg [1:0]  sw_reg;
12 reg        sw_change;
13 reg        rst_sync;

```

```

14 reg [19:0] rst_cnt;
15 reg      clear;
16 // ¼ SW ±秒
17 always @(posedge clk or negedge rst_n) begin
18     if(!rst_n) begin
19         sw_reg    <= 2'd0;
20         sw_change <= 1'b0;
21     end else begin
22         sw_reg    <= sw;
23         sw_change <= (sw_reg != sw && sw != 2'd0);
24     end
25 end
26
27 // ¾ ³ SW ±秒 ; -µ -
28 always @(posedge clk or negedge rst_n) begin
29     if(!rst_n) begin
30         rst_sync <= 1'b1;
31         rst_cnt  <= 20'd0;
32     end
33     else if(sw_change) begin
34         rst_sync <= 1'b0;
35         rst_cnt  <= 20'd0;
36     end
37     else if(rst_cnt < 20'd100_000) begin
38         rst_cnt  <= rst_cnt + 1'b1;
39         rst_sync <= 1'b0;
40     end
41     else begin
42         rst_sync <= 1'b1;
43     end
44 end
45
46 // ===== »- g¿ =====
47 Oled_Top Oled_Top_u(
48     .clk      (clk),
49     .rst      (rst_sync), // SW ±秒 »K
50     .sw       (sw),
51     .clear_key (sw[1] | sw[0]),
52     .on_key   (1'b0), // ½ µ
53     .vcc      (vcc),

```



```
54     .scl      (scl),  
55     .sda      (sda)  
56 );  
57  
58 endmodule
```

Listing 10: topcontrol.v ——OLED 顶层封装（SW 检测与自动复位）